# UADFormer: A Transformer-Based Deep Learning Method for User Activity Detection in Cell-Free Massive MIMO Systems

Zheng Sheng, Pengcheng Zhu, *Member, IEEE*, and Xiaohu You, *Fellow, IEEE*

***Abstract*— Grant-free random access is a critical enabling technology for massive ultra-reliable and low-latency communications (mURLLC), and user activity detection (UAD), determining which users are active based on received signals, is indispensable in grant-free access. The cell-free massive multiple-input multiple-output (MIMO) system, providing macro diversity and supporting more users, is an architecture suitable for mURLLC. This paper investigates the UAD problem with changeable pilot sequences in cell-free massive MIMO systems, and proposes deep learning based UADFormer, where users are assigned to access points (APs) for detection using user grouping algorithm and transformer-based neural networks (NNs) in APs are employed to output user activity state by exploiting the correlation between received signals and pilot sequences. For users detected by multiple APs, a fusion NN based on transfer learning in the central processing unit is utilized to enhance UAD accuracy. Additionally, considering limited computational and storage resources in APs, sparse attention mechanism is used in NNs to reduce computational complexity and residual attention score is employed to improve NN performance. Simulation results show that UADFormer outperforms other schemes in various evaluation metrics and reduces computational complexity.**

***Index Terms*— User activity detection (UAD), cell-free massive multiple-input multiple-output (MIMO), deep learning, transformer, ultra-reliable and low-latency communications (URLLC).**

## I. Introduction

RECENTLY, massive ultra-reliable and low-latency communications (mURLLC) [1], also known as critical massive machine type communications (mMTC) [2], which integrates two key use cases in the fifth generation (5G) mobile communication, URLLC and mMTC [3], has attracted considerable attention from researchers. Specifically, URLLC strives to provide highly reliable connectivity (more than $99.999\%$) with extremely low latency (less than 1ms) and mMTC pursues scalable connectivity ($10^6$ devices/km$^2$) [4], [5]. In fact, for many not-so-distant Internet-of-Things (IoT) applications, such as robotics in large factory networks and massive network of autonomous vehicles, both highly reliable wireless connectivity with extremely low latency and scalable connectivity are indispensable, and research on related enabling technologies is promising [6].

Grant-free access is one of the related enabling technologies and has received much attention [7], [8]. Compared with grant-based access adopted by long term evolution (LTE) standard, which brings heavy signaling overhead and large latency due to its four-way handshake procedure in the scheduling process, grant-free access can effectively reduce signaling overhead and access latency since user devices can directly access the base station (BS) by skipping the handshake based grant acquisition process [9], which is highly applicable to mURLLC. As each user device transmits information to the BS without scheduling, the BS cannot directly distinguish which users have transmitted information. Therefore, user activity detection (UAD), i.e., identifying users transmitting information, is significant and indispensable in grant-free systems [10].

The cell-free massive multiple-input multiple-output (MIMO) system, where many distributed access points (APs) connected to the central processing unit (CPU) serve users utilizing the same time-frequency resources, is considered a highly promising architecture. Compared to the centralized cellular system, the cell-free system is more suitable for mURLLC applications due to the following three reasons. First, since APs in cell-free systems usually possess the data processing capability [11] and there are massive users in the mURLLC application scenario, users can be grouped and each AP is only responsible for detecting a group of users. Compared with the computational complexity of detecting all users employing one BS in the centralized cellular system, the complexity of each AP in the cell-free system is reduced. Moreover, APs simultaneously detect the activity states of users assigned to them, i.e., through parallel processing, thereby further reducing processing latency. Second, UAD in the cell-free system can support more users than that in the centralized cellular system. When the number of users increases to a very large value, due to limited storage and computing resources in one BS, UAD in the centralized cellular system will reach a bottleneck, i.e., the base station cannot detect so many users accurately. In the cell-free system, more APs can be deployed and the number of users detected by each AP will not increase, which

Received 17 March 2024; revised 7 August 2024; accepted 24 September 2024. Date of publication 7 October 2024; date of current version 12 December 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB2900300 and in part by the National Natural Science Foundation of China under Grant 62171126. The associate editor coordinating the review of this article and approving it for publication was G. Chen. *(Corresponding author: Pengcheng Zhu.)*

Zheng Sheng is with the National Mobile Communications Research Laboratory, Southeast University, Nanjing 210096, China (e-mail: shengzheng@seu.edu.cn).

Pengcheng Zhu and Xiaohu You are with the National Mobile Communications Research Laboratory, Southeast University, Nanjing 210096, China, and also with the Purple Mountain Laboratories, Nanjing 211111, China (e-mail: p.zhu@seu.edu.cn; xhyu@seu.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TWC.2024.3469970.

Digital Object Identifier 10.1109/TWC.2024.3469970

ensures the accurate detection. Third, the cell-free system, where multiple APs jointly serve users, can provide macro diversity [12], which brings robust connectivity and enhances communication reliability. In general, UAD in the cell-free system is worth studying [13].

To deal with the UAD problem, traditional methods include compressed sensing (CS) based methods and covariance based methods [14], [15]. Exploiting sporadic active users, CS based methods, such as approximate message passing (AMP) [16], regularization based sparse optimization [17] and Bayesian sparse recovery [18], are employed to address joint UAD and channel estimation problems. When employing CS based methods, instantaneous channel state information (CSI) is required, which is not feasible in practice. Thus, covariance based methods using sample covariance matrix of the received signal, which exploits statistical CSI rather than instantaneous CSI, has been designed [19], [20]. In fact, [21] and [22] have demonstrated that covariance based methods can achieve similar performance with a much shorter length of pilot sequences compared with CS based methods. Although covariance based methods outperform CS based methods, in many existing works such as [19] and [20], it utilizes block coordinate descent methods to solve high-dimensional non-convex optimization problems where each user's activity state is updated iteratively. This results in large processing latency and violates requirements of mURLLC.

In recent years, with the rise of deep learning (DL) [23], many researchers have proposed DL based methods to tackle the UAD problem in centralized systems. DL based methods train a deep neural network (NN) to approximate the mapping from known information (including received signals and user pilot sequences) to activity states of all users. Compared with traditional methods, the deep NN can directly output all users' activity states without updating iteratively in the inference phase, hence reducing the processing latency. In [24], authors have proposed a model-driven DL algorithm which employs the NN to imitate AMP. In [9], authors have designed a data-driven DL approach to directly output the activity state with received signals as input in the inference phase. However, in these two papers, user pilot sequences are fixed and only received signals are changeable input of NNs. NNs need to be retrained if there is any change in the pilot sequence of any user, which are unsuitable for changing environments. Different from the previous two papers, [25] has presented a novel deep NN named heterogeneous transformer (HT), where inputs include received signals and changeable pilot sequences, to deal with the UAD problem in centralized systems. Transformer was initially applied in machine translation, known for its ability to explore relationships between long sequences [26]. HT determines user activity state by uncovering correlations between received signals and changeable pilot sequences, and there is no need to retrain the NN as with the previous two methods when pilot sequences change. However, it is necessary to conduct transmission power control to eliminate impacts of large-scale fading. Otherwise, due to different large-scale fading coefficients, received signals from different users exhibit significant differences in magnitude, and we will fail to train the NN. Unfortunately, in cell-free systems, transmission power control cannot remove effects of large-scale fading due to the existence of multiple APs, which poses a challenge to applying HT in cell-free systems.

The UAD in cell-free systems has attracted much attention and there have been some research works about it. In [27], authors have proposed a CS based scheme to conduct joint channel estimation and UAD in cell-free systems with coarse user location information. Authors in [28] have designed a covariance based approach with orthogonal pilot sequences to address the UAD problem in cell-free systems. In [29], exploiting characteristics of cell-free systems, authors have deployed NNs with fixed pilot sequences in space expansion units and trained a NN in CPU based on transfer learning to integrate judgments from all space expansion units to output the final result, thereby enhancing detection accuracy. However, existing methods for cell-free systems cannot effectively handle the UAD problem with changeable pilot sequences. Specifically, when pilot sequences change, traditional methods in existing works require solving new non-convex optimization problems iteratively again and DL based approaches in current papers need to retrain NNs, both causing large processing latency.

Inspired by above observations, this paper proposes a DL based method named UADFormer to solve the UAD problem with changeable pilot sequences in cell-free systems, and answers two main questions: (1) how to successfully train a transformer-based NN in cell-free systems without eliminating impacts of large-scale fading; (2) how to effectively improve the accuracy of UAD by leveraging advantages (APs with processing capability) and considering limitations (limited computational and storage resources in APs) of cell-free systems. The novelty of our proposed UADFormer is mainly reflected in the following two points: (1) distributed architecture, i.e., deploying NNs based on lite transformer in APs (using sparse attention mechanism and residual attention score) and a fusion NN based on transfer learning in the CPU; (2) considering UAD as multiple binary classification problems and an imbalanced multi-label classification problem, thereby choosing the most suitable loss function to accelerate the NN training. The major contributions of this paper are summarized as follows.

1) We consider the UAD problem as multiple distinct imbalanced binary classification problems and an imbalanced multi-label classification problem, choosing binary cross-entropy (BCE) and weighted binary cross-entropy (WBCE) as loss functions for the former, and selecting zero-bounded log-sum-exp & pairwise rank-based (ZLPR) as the loss function for the latter. By comparing their performance, we choose the most suitable one to achieve high classification accuracy and good balance.
2) We propose a user grouping algorithm based on large-scale fading coefficients, where each AP is responsible for detecting users with higher large-scale fading coefficients rather than all users. Since received signals from users with higher large-scale fading coefficients are much stronger than those from users with lower coefficients, received signals from users with lower coefficients can be viewed as very weak interference and we can

successfully train the NN without eliminating effects of large-scale fading.

3) We create the UADFormer consisting of NNs in APs and the NN in the CPU. In each AP, the transformer-based NN, where inputs consist of the received signal and changeable pilot sequences of users assigned to this AP, outputs the user activity probability by employing transformer's ability to explore the correlation between the received signal and pilot sequences.
4) We utilize sparse attention mechanism to reduce computational and storage complexity considering limited computational and storage resources in APs. Specifically, we only calculate the attention score between each user's pilot sequence and the received signal, without computing attention scores between this user's pilot sequence and other users' pilot sequences. Additionally, the residual attention score is used to enhance the performance of NNs deployed in APs.
5) We design the fusion module based on transfer learning in the CPU for users detected by multiple APs. It takes judgments about the same user's activity state from multiple APs as inputs and outputs the final detection result, which improves the accuracy of detection.

The rest of this paper is organized as follows. The system model is introduced in Section II, and problem formulation is shown in Section III. In Section IV, we design our user grouping algorithm. A novel DL approach named UADFormer is proposed in Section V and its complexity is also analyzed in Section V. Simulation results are provided in Section VI, and Section VII concludes this paper.

## II. System Model

In the cell-free massive MIMO system under consideration, $M$ APs each equipped with $N$ antennas and $K$ single antenna IoT users are located randomly in a large area. Each AP possesses data processing capability and all APs are connected to a CPU via fronthaul links. The channel $\mathbf{g}_{m,k} \in \mathbb{C}^{N\times 1}$ between AP $m$ and user $k$ can be expressed as

$$\mathbf{g}_{m,k} = \sqrt{\lambda_{m,k}}\mathbf{h}_{m,k}, \tag{1}$$

where $\lambda_{m,k}$ represents a large-scale fading coefficient and $\mathbf{h}_{m,k}$ represents the small-scale fading vector. We consider a rich scattering environment, so $\mathbf{h}_{m,k}$ can be modeled as $\mathcal{CN}(\mathbf{0}, \mathbf{I}_N)$.

In the UAD problem, each user is assigned a unique uplink pilot sequence $\mathbf{s}_k \in \mathbb{C}^{L\times 1}$ of length $L$ and user $k$ will transmit $\mathbf{s}_k$ if it is active. The received signals $\mathbf{Y}_m$ at AP $m$ are expressed as

$$\begin{aligned}\mathbf{Y}_m &= \sum_{k=1}^{K} x_k\sqrt{q_k}\mathbf{s}_k\mathbf{g}_{m,k}^T + \mathbf{W}_m \\ &= \mathbf{S}\mathbf{X}\sqrt{\mathbf{Q}}\sqrt{\mathbf{\Lambda}_m}\mathbf{H}_m + \mathbf{W}_m,\end{aligned} \tag{2}$$

where $x_k \in \{0,1\}$ is the activity indicator and $x_k = 1$ if user $k$ is active (otherwise, $x_k = 0$), $q_k$ is the transmission power of user $k$, $\mathbf{S} = [\mathbf{s}_1, \ldots, \mathbf{s}_K] \in \mathbb{C}^{L\times K}$, $\mathbf{X} = \operatorname{diag}\{x_1, \ldots, x_K\} \in \mathbb{C}^{K\times K}$, $\mathbf{Q} = \operatorname{diag}\{q_1, \ldots, q_K\} \in \mathbb{C}^{K\times K}$, $\mathbf{\Lambda}_m = \operatorname{diag}\{\lambda_{m,1}, \ldots, \lambda_{m,K}\} \in \mathbb{C}^{K\times K}$, $\mathbf{H}_m = [\mathbf{h}_{m,1}, \ldots, \mathbf{h}_{m,K}]^T \in \mathbb{C}^{K\times N}$, and $\mathbf{W}_m \in \mathbb{C}^{L\times N}$ represents the Gaussian noise at AP $m$ and each element follows $\mathcal{CN}(0, \sigma_m^2)$.

Considering the entire system, the signals received by all APs can be written as

$$\mathbf{Y} = \mathbf{S}\mathbf{X}\sqrt{\mathbf{Q}}\mathbf{G} + \mathbf{W}, \tag{3}$$

where $\mathbf{Y} = [\mathbf{Y}_1, \ldots, \mathbf{Y}_M] \in \mathbb{C}^{L\times MN}$ denotes signals received by all APs, $\mathbf{G} = \left[\sqrt{\mathbf{\Lambda}_1}\mathbf{H}_1, \ldots, \sqrt{\mathbf{\Lambda}_M}\mathbf{H}_M\right] \in \mathbb{C}^{K\times MN}$ represents the channel between all APs and all users, and $\mathbf{W} = [\mathbf{W}_1, \ldots, \mathbf{W}_M] \in \mathbb{C}^{L\times MN}$ represents the noise at all APs.

## III. Problem Formulation

In practical applications, IoT user devices are typically stationary, and as a result, their large-scale fading coefficients remain relatively constant [19], [20], [25]. In this paper, we assume that the large-scale fading coefficients of users remain constant and can be estimated through conventional methods[1] [30].

### A. Problem Formulation From DL Perspective

The UAD aims to determine the activity state of each user with known received signals and pilot sequence assigned to each user. Different from CS based methods requiring instantaneous CSI and covariance based approaches requiring iterative solving, DL, utilizing the NN, strives to discover a mapping function that minimizes the difference between the output and the ground truth as much as possible when provided with the input. Without the need for instantaneous CSI and iterative solving, DL is adopted by us to deal with the UAD problem in this paper.

Specifically, the NN takes the received signal matrix $\mathbf{Y}$ and the pilot sequence matrix $\mathbf{S}$ as inputs and generates the predicted user activity state vector $\hat{\mathbf{x}} = [\hat{x}_1, \ldots, \hat{x}_K]^T$ as the output. With the actual user activity state vector $\mathbf{x} = [x_1, \ldots, x_K]^T$, we aspire to find the optimal mapping function that ensures $\hat{\mathbf{x}}$ aligns as closely as possible with $\mathbf{x}$. The problem of designing this mapping function can be formulated as:

$$\begin{aligned}&\min_{\mathbf{\Theta}} \quad \mathbb{E}\left\{\mathcal{L}\left(\mathbf{x}, \hat{\mathbf{x}}\right)\right\}, \\ &\text{s.t. } \hat{\mathbf{x}} = f_{\mathbf{\Theta}}\left(\mathbf{Y}, \mathbf{S}\right),\end{aligned} \tag{4}$$

where $f_{\mathbf{\Theta}}\left(\mathbf{Y}, \mathbf{S}\right)$ represents the mapping function that maps $\mathbf{Y}$ and $\mathbf{S}$ to $\hat{\mathbf{x}}$, $\mathbf{\Theta}$ denotes parameters of the NN including weights and biases, and $\mathcal{L}\left(\mathbf{x}, \hat{\mathbf{x}}\right)$ serves as a function that quantifies the disparity between $\mathbf{x}$ and $\hat{\mathbf{x}}$.

### B. Classification Problems From Two Perspectives

Since $\mathbf{x}$ is a binary vector of length $K$, we naturally associate it with the classification task in the field of DL, and the UAD problem can be regarded as $K$ distinct binary classification problems or a multi-label classification problem.

[1]It is worth noting that the first step of user activity detection is to connect all users to the network, then we estimate the large-scale fading coefficient between each user and each AP, and subsequently we can group all users into groups employing our proposed user grouping algorithm based on known large-scale fading coefficients.

*1) $K$ Distinct Binary Classification Problems:* In the UAD problem, each user is either active or inactive, so determining the activity state of one user can be viewed as a binary classification problem. As there are $K$ distinct users in the system, identifying activity states of all users can be considered as $K$ independent binary classification problems.

For classification problems, NNs' outputs are typically probabilities and final results are obtained by comparing probabilities with thresholds. For the UAD problem, employing $\hat{\mathbf{p}} = [\hat{p}_1, \ldots, \hat{p}_K]^T$ to represent predicted activity probabilities of all $K$ users, the final result is

$$\hat{x}_i = \mathbb{I}(\hat{p}_i > \tau), \tag{5}$$

which means that if the $i$-th element of $\hat{\mathbf{p}}$ is greater than predefined threshold $\tau$, the $i$-th element of $\hat{\mathbf{x}}$ is set to 1, otherwise set to 0.

We utilize $\mathcal{T}$ to denote the training data set, where the $i$-th sample is composed of $\left(\mathbf{Y}^{(i)}, \mathbf{S}^{(i)}, \mathbf{x}^{(i)}\right)$ and $\mathbf{x}^{(i)}$ denotes the ground truth user activity state. For the binary classification problem, the loss function used to train NNs is typically BCE function, which is expressed as follows:

$$\mathcal{L}_{\text{BCE}} = -\sum_{i=1}^{|\mathcal{T}|}\sum_{k=1}^{K}\left(x_k^{(i)}\log\hat{p}_k^{(i)} + (1-x_k^{(i)})\log(1-\hat{p}_k^{(i)})\right), \tag{6}$$

where $|\mathcal{T}|$ represents the number of samples in $\mathcal{T}$ and $\log(\cdot)$ denotes natural logarithm.

Nevertheless, in the UAD problem, due to sporadic traffics of mMTC, only a small percentage of users are active. Thus, UAD is an imbalanced classification problem and BCE is no longer suitable, as it pays more attention to more numerous inactive users and overlooks less numerous active users, resulting in overfitting to inactive class.

To mitigate overfitting, we have fine-tuned weights of BCE based on the number of active users $K_a$, increasing the weight of active class to $K_a/K$ and decreasing the weight of inactive class to $(K-K_a)/K$.[2] The modified loss function called WBCE is written as

$$\mathcal{L}_{\text{WBCE}} = -2\sum_{i=1}^{|\mathcal{T}|}\sum_{k=1}^{K}\left(\frac{K-K_a}{K}x_k^{(i)}\log\hat{p}_k^{(i)} + \frac{K_a}{K}(1-x_k^{(i)})\log(1-\hat{p}_k^{(i)})\right). \tag{7}$$

*2) A Multi-Label Classification Problem:* From a holistic perspective, the UAD problem can also be considered as a multi-label classification problem with $K$ labels, where each element of the output $\hat{\mathbf{x}}$ is viewed as a label indicating whether the corresponding user is active. Since only a small fraction of users are active, this problem is an imbalanced multi-label classification task.

To address imbalanced multi-label classification problems, recently, a novel loss function named ZLPR has been proposed [31]. Compared with WBCE requiring $K_a$ that is usually unknown in practical scenarios, ZLPR can be employed to train NNs when $K_a$ is unknown. The expression of ZLPR is

$$\mathcal{L}_{\text{ZLPR}} = \sum_{i=1}^{|\mathcal{T}|}\left(\log\left(1+\sum_{k=1}^{K}x_k^{(i)}e^{-a_k^{(i)}}\right) + \log\left(1+\sum_{k=1}^{K}(1-x_k^{(i)})e^{a_k^{(i)}}\right)\right), \tag{8}$$

where $[a_1^{(i)}, \ldots, a_K^{(i)}]$ represents the logit vector of the $i$-th sample. The relationship between $\hat{p}_k^{(i)}$ and $a_k^{(i)}$ is

$$\hat{p}_k^{(i)} = \text{sigmoid}(a_k^{(i)}) = \frac{1}{1+e^{-a_k^{(i)}}}. \tag{9}$$

Since $\mathbf{x}^{(i)}$ is a binary vector, (8) can be simplified as

$$\mathcal{L}_{\text{ZLPR}} = \sum_{i=1}^{|\mathcal{T}|}\left(\log\left(1+\sum_{u\in\Delta_{pos}^{(i)}}e^{-a_u^{(i)}}\right) + \log\left(1+\sum_{v\in\Delta_{neg}^{(i)}}e^{a_v^{(i)}}\right)\right), \tag{10}$$

where $\Delta_{pos}^{(i)} = \{u|x_u^{(i)} = 1\}$ and $\Delta_{neg}^{(i)} = \{v|x_v^{(i)} = 0\}$. As the log-sum-exp operator serves as a smooth approximation to the maximum operator, which is proved in Appendix B, we have

$$\log\left(1+\sum_{k=1}^{K}e^{a_k^{(i)}}\right) \approx \max(0, a_1^{(i)}, \ldots, a_K^{(i)}), \tag{11}$$

and (10) can be approximated as

$$\mathcal{L}_{\text{ZLPR}} \approx \sum_{i=1}^{|\mathcal{T}|}\left(\max\left(0, \underbrace{-a_u^{(i)}, \ldots}_{u\in\Delta_{pos}^{(i)}}\right) + \max\left(0, \underbrace{a_v^{(i)}, \ldots}_{v\in\Delta_{neg}^{(i)}}\right)\right). \tag{12}$$

It can be observed from (12) that the minimum value of ZLPR is approximately 0 when $a_u^{(i)} > 0$ for all active users and $a_v^{(i)} < 0$ for all inactive users, where the detection success rate is 100% with $\tau = 0.5$. Moreover, in both cases $a_u^{(i)} < 0$ and $a_v^{(i)} > 0$, ZLPR will increase equally, which avoids overfitting to inactive users.

## IV. User Grouping Algorithm

Authors in [25] have proposed HT to deal with the UAD problem in centralized systems by conducting transmission power control to eliminate impacts of large-scale fading. However, their method cannot be directly applied to cell-free systems since transmission power control cannot eliminate effects of large-scale fading in cell-free systems. In mathematical terms, there does not exist a set $\{q_1, \ldots, q_K\}$ such that the equation $q_1\lambda_{m,1} = \ldots = q_K\lambda_{m,K}$ for all $m$ holds true. The large-scale fading coefficients of users with higher large-scale fading coefficients may differ by several orders of magnitude from those of users with lower large-scale fading coefficients. In extreme cases, the AP's received signals from

[2]The number of active users $K_a$ is unknown in our paper but it can be estimated in reality through the method in Appendix A using statistical channel state information.

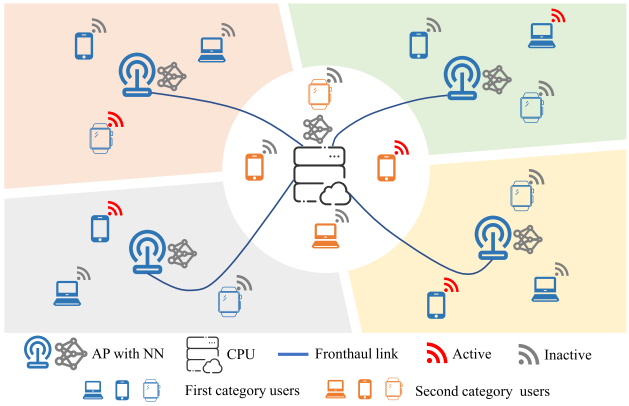



Fig. 1. Proposed user grouping diagram where blue users are detected by one AP and orange users are detected by multiple APs.

very distant users are the same magnitude as noise, and it is nearly impossible to determine activity states of these very distant users accurately.

Since it is difficult to accurately determine activity states of users with lower large-scale fading coefficients, why not have each AP only assess the activity states of users with higher large-scale fading coefficients rather than all users? In this detection approach, it is essential to determine which users have higher large-scale fading coefficients with the AP. The most straightforward method is to assign each user to the AP with the highest large-scale fading coefficient among all APs. However, if a user has relatively high large-scale fading coefficients with two or more APs, its signal will cause significant interference for APs with relatively high large-scale fading coefficients but not responsible for detecting it, thereby reducing the accuracy of UAD. To address this problem, we divide users into two categories: those who have the high large-scale fading coefficient only with one AP, and those who have relatively high large-scale fading coefficients with two or more APs, as illustrated in Fig. 1. The first category is detected by single AP, while the second category is detected by multiple APs.

The main idea of our proposed user grouping algorithm is as follows. Each user is initially assigned to the AP having the highest large-scale fading coefficient with it among all APs for detection. Subsequently, it is determined whether the second highest large-scale fading coefficient is close to the highest large-scale fading coefficient, as well as whether the number of users assigned to each AP has reached its limit. This process continues iteratively until all users and APs have been traversed. Considering large-scale fading coefficients between users and APs, as well as the load of each AP, we propose our user grouping algorithm shown in Algorithm 1 in detail.

The large-scale fading coefficients between $K$ users and $M$ APs are stored in the matrix $\mathbf{\Lambda} \in \mathbb{R}^{K\times M}$ and $\mathbf{\Lambda}(k,m)$ represents $\lambda_{m,k}$, i.e., the large-scale fading coefficient between user $k$ and AP $m$. The user set detected by AP $m$ is denoted as $\mathcal{U}_m$. $\epsilon$ represents the criteria for judging whether $\lambda_{m,k}$ and maximum large-scale fading coefficient $\lambda_{m_k^{(1)},k}$ for user $k$ among all $M$ APs are close, and $\xi$ denotes the maximum number of users that each AP can detect. Considering "argmax" operation and adding the user into corresponding user set as a basic operation, the computational complexity of Algorithm 1 is $\mathcal{O}(MK)$, and even in the worst-case scenario, Algorithm 1 can converge after $MK$ basic operations.

**Algorithm 1** Proposed user grouping algorithm

**Input:** $\mathbf{\Lambda}, \mathcal{U}_m = \emptyset \ (m = 1, \ldots, M)$
**for** $k = 1 : K$ **do**
  $m_k^{(1)} = \underset{m\in\{1,\ldots,M\}}{\operatorname{argmax}} \ \mathbf{\Lambda}(k,m)$
  $\mathcal{U}_{m_k^{(1)}} = \mathcal{U}_{m_k^{(1)}} \cup \{k\}$
**end for**
**for** $i = 2 : M$ **do**
  **for** $k = 1 : K$ **do**
    $m_k^{(i)} = \underset{m\in\{1,\ldots,M\}\setminus\{m_k^{(1)},\ldots,m_k^{(i-1)}\}}{\operatorname{argmax}} \mathbf{\Lambda}(k,m)$
    **if** $\mathbf{\Lambda}(k,m_k^{(1)}) - \mathbf{\Lambda}(k,m_k^{(i)}) < \epsilon$ & $|\mathcal{U}_{m_k^{(i)}}| < \xi$ **then**
      $\mathcal{U}_{m_k^{(i)}} = \mathcal{U}_{m_k^{(i)}} \cup \{k\}$
    **end if**
  **end for**
**end for**
**Output:** $\mathcal{U}_m \ (m = 1, \ldots, M)$

## V. UADFormer: A Novel Deep Learning Approach Based on Lite Transformer and Transfer Learning

To address the UAD problem in cell-free systems, we have proposed a novel DL approach named UADFormer based on characteristics of the UAD problem and the cell-free system, which is shown in Fig. 2. The UADFormer consists of two parts: an NN in each AP based on lite transformer, and an NN in the CPU based on transfer learning.

### A. NN in Each AP Based on Lite Transformer

Different from multilayer perceptrons (MLPs) and convolutional neural networks (CNNs) not fully exploiting characteristics of the UAD problem, transformer can explore the correlation between $\mathbf{Y}_m$ and $\mathbf{S}_m$ to identify the user activity state. Specifically, in (2), each column of $\mathbf{Y}_m$ can be regarded as a linear combination of $\mathbf{s}_k \ (k = 1, \ldots, K)$ and coefficients are related to $\mathbf{x}$. In transformer, $\mathbf{Y}_m$ and $\mathbf{S}_m$ are respectively represented in high-dimensional space, and the correlation between the representation of $\mathbf{Y}_m$ and the representation of each user's pilot sequence in $\mathbf{S}_m$ is calculated. The activity state of each user detected by AP $m$ is determined by the magnitude of correlation. Considering limited computational and storage resources in APs, we propose a lite transformer using sparse attention mechanism with fewer parameters and lower computational complexity compared with the standard transformer.

Since NN structures of different APs are similar, we will take AP $m$ as an example to illustrate the NN. For notational simplicity, we have omitted the subscript $m$ starting from the output of preprocessing module in this subsection.

*1) Preprocessing and Linear Projection:* The structure of preprocessing and linear projection block is shown in Fig. 3 in detail. This module takes $\mathbf{Y}_m$ and $\mathbf{S}_m$ as inputs and outputs $\{\mathbf{z}_{m,n}\}_{n=0}^{K_m}$, which are representations of inputs after this module.

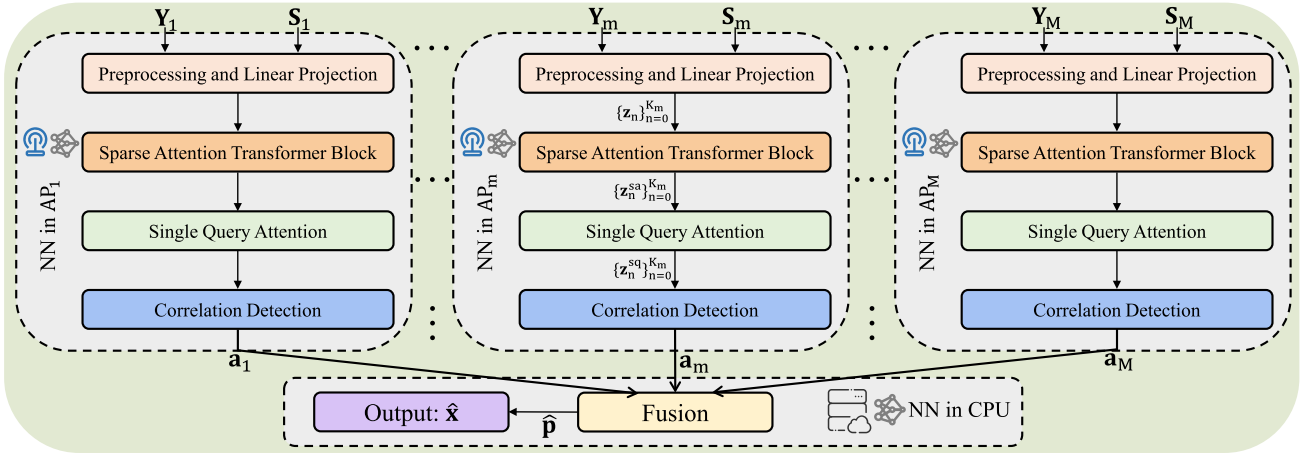



Fig. 2. The overview of UADFormer. $\mathbf{Y}_m$ represents the received signal of AP $m$. $\mathbf{S}_m = [\tilde{\mathbf{s}}_{m,1}, \ldots, \tilde{\mathbf{s}}_{m,K_m}]$ consists of pilot sequences of users detected by AP $m$, where $K_m$ denotes the number of users detected by AP $m$. The logit vector of users detected by AP $m$ is denoted by $\mathbf{a}_m$ and the activity probability vector of all users in the cell-free system is represented by $\hat{\mathbf{p}}$. By comparing $\hat{\mathbf{p}}$ with the threshold, the final output $\hat{\mathbf{x}}$ can be obtained.

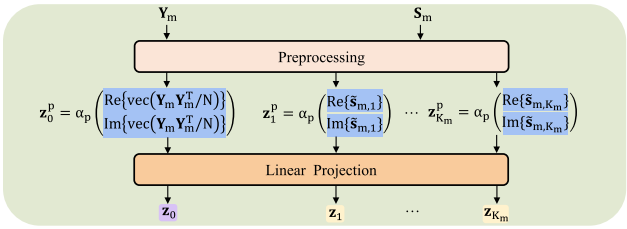



Fig. 3. The detailed structure of preprocessing and linear projection block, where the projection matrix of $\mathbf{z}_0$ is different from others.

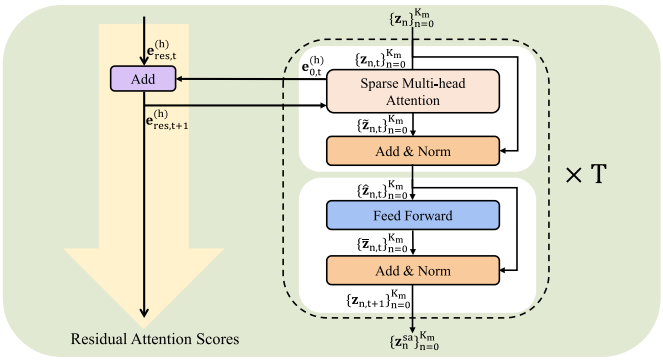



Fig. 4. The sparse attention transformer block, consisting of $T$ sparse attention transformer encoders and residual attention scores.

As NNs can usually only handle real numbers while pilot sequences and received signals are complex, it is necessary to extract their real and imaginary parts for processing. Inspired by covariance based method, we utilize $\mathbf{Y}_m\mathbf{Y}_m^H/N$ instead of $\mathbf{Y}_m$ as the input to the linear projection, enabling NNs to solve problems with different numbers of antennas. The output of $\mathbf{Y}_m$ after passing through preprocessing module can be represented as

$$\mathbf{z}_0^{\mathrm{p}} = \alpha_{\mathrm{p}} \left[ \mathcal{R}\left\{ \mathrm{vec}\left( \frac{\mathbf{Y}_m \mathbf{Y}_m^H}{N} \right) \right\}^T, \mathcal{I}\left\{ \mathrm{vec}\left( \frac{\mathbf{Y}_m \mathbf{Y}_m^H}{N} \right) \right\}^T \right]^T, \tag{13}$$

where $\mathbf{z}_0^{\mathrm{p}} \in \mathbb{R}^{2L^2}$ and $\alpha_{\mathrm{p}}$ is a tuning hyperparameter of preprocessing and linear projection module which scales up data values for training NNs. $\mathcal{R}\{\cdot\}$ and $\mathcal{I}\{\cdot\}$ denote real part and imaginary part, respectively. The column vectorization of a matrix is represented by $\mathrm{vec}(\cdot)$. Similarly, $\{\mathbf{z}_n^{\mathrm{p}}\}_{n=1}^{K_m} \in \mathbb{R}^{2L}$, which is the output of $\tilde{\mathbf{s}}_{m,n}$, can be expressed as

$$\mathbf{z}_n^{\mathrm{p}} = \alpha_{\mathrm{p}} \left[ \mathcal{R}\left\{\tilde{\mathbf{s}}_{m,n}\right\}^T, \mathcal{I}\left\{\tilde{\mathbf{s}}_{m,n}\right\}^T \right]^T, \ n = 1, \ldots, K_m. \tag{14}$$

Then, $\mathbf{z}_n^{\mathrm{p}}$ will be sent to linear projection module. Since the received signal and pilot sequences have different physical meanings, two different trainable projection matrices $\mathbf{W}_{\mathrm{Y}}^{\mathrm{p}} \in \mathbb{R}^{d\times 2L^2}$ and $\mathbf{W}_{\mathrm{S}}^{\mathrm{p}} \in \mathbb{R}^{d\times 2L}$ are used to project $\mathbf{z}_0^{\mathrm{p}}$ and $\{\mathbf{z}_n^{\mathrm{p}}\}_{n=1}^{K_m}$, respectively. The outputs are

$$\mathbf{z}_n = \begin{cases} \mathbf{W}_{\mathrm{Y}}^{\mathrm{p}} \mathbf{z}_0^{\mathrm{p}} & n = 0 \\ \mathbf{W}_{\mathrm{S}}^{\mathrm{p}} \mathbf{z}_n^{\mathrm{p}} & n = 1, \ldots, K_m, \end{cases} \tag{15}$$

where $\{\mathbf{z}_n\}_{n=0}^{K_m} \in \mathbb{R}^d$ and $d$ denotes the dimension of linear projection module.

*2) Sparse Attention Transformer Block:* The sparse attention transformer block is the core module of UADFormer, which consists of $T$ sparse attention transformer encoders and residual attention scores. Fig. 4 illustrates the detailed structure and each module will be introduced below.

Considering limited computational and storage resources in APs, the sparse multi-head attention module is designed to replace the standard multi-head attention, and the relationship between the input $\{\mathbf{z}_{n,t}\}_{n=0}^{K_m} \in \mathbb{R}^d$ and the output $\{\tilde{\mathbf{z}}_{n,t}\}_{n=0}^{K_m} \in \mathbb{R}^d$ of sparse multi-head attention is

$$\{\tilde{\mathbf{z}}_{n,t}\}_{n=0}^{K_m} = \mathrm{SMA}\left(\{\mathbf{z}_{n,t}\}_{n=0}^{K_m}\right), \tag{16}$$

where SMA represents the function of the sparse multi-head attention module. The overall architecture of the sparse multi-head attention module is illustrated in Fig. 5. In order to fully explore the correlation between the received signal and each user's pilot sequence, we employ $H$ attention heads to provide greater freedom for NNs. The output of each attention head will be weighted and summed to obtain the final output. For all attention heads in the $t$-th transformer encoder, the input is $\{\mathbf{z}_{n,t}\}_{n=0}^{K_m}$, and next, we will take the $h$-th head as an example to explain the detailed structure of attention heads.

In the standard attention mechanism, all input tokens are projected as queries because there is correlation between each

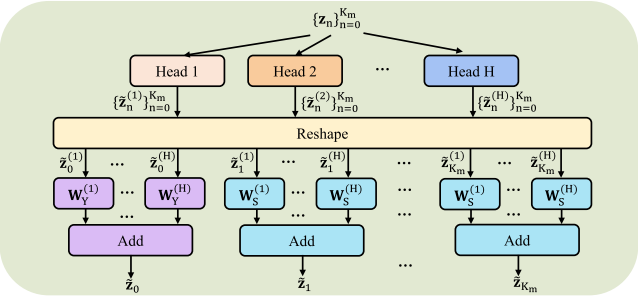



Fig. 5. The overall architecture of the sparse multi-head attention module. For notational simplicity, we have omitted the subscript $t$ in the figure.

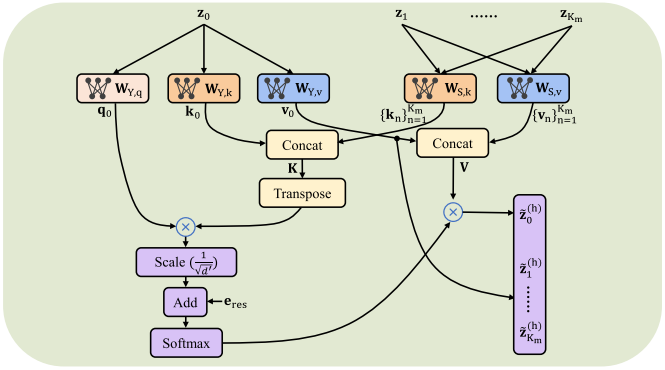



Fig. 6. The structure of sparse attention mechanism in the $h$-th attention head, where $h$ in the superscript and $t$ in the subscript are omitted for notational simplicity expect the final output.

token. However, in the UAD problem, only the received signal is related to the pilot sequence of each user while there is no correlation between user pilot sequences. Thus, in our sparse attention, only $\mathbf{z}_{0,t}$ is projected as the query and the query projection matrix is $\mathbf{W}^{(h)}_{\mathrm{Y,q},t}$. Different from the input tokens with the same physical meaning in the standard attention mechanism, the received signal and user pilot sequences have distinct physical meanings. Therefore, independent learnable projection matrices are assigned to $\mathbf{z}_{0,t}$ and $\{\mathbf{z}_{n,t}\}_{n=1}^{K_m}$. Specifically, $\mathbf{W}^{(h)}_{\mathrm{Y,k},t}$ and $\mathbf{W}^{(h)}_{\mathrm{Y,v},t}$ represent the key projection matrix and value projection matrix for $\mathbf{z}_{0,t}$ respectively, while $\mathbf{W}^{(h)}_{\mathrm{S,k},t}$ and $\mathbf{W}^{(h)}_{\mathrm{S,v},t}$ separately denote the key projection matrix and value projection matrix for $\{\mathbf{z}_{n,t}\}_{n=1}^{K_m}$. With independent learnable projection matrices, the projection result can be expressed as:

$$\mathbf{q}^{(h)}_{0,t} = \mathbf{W}^{(h)}_{\mathrm{Y,q},t}\mathbf{z}_{0,t}, \tag{17}$$

$$\mathbf{k}^{(h)}_{n,t} = \begin{cases} \mathbf{W}^{(h)}_{\mathrm{Y,k},t}\mathbf{z}_{0,t} & n=0 \\ \mathbf{W}^{(h)}_{\mathrm{S,k},t}\mathbf{z}_{n,t} & n=1,\ldots,K_m, \end{cases} \tag{18}$$

$$\mathbf{v}^{(h)}_{n,t} = \begin{cases} \mathbf{W}^{(h)}_{\mathrm{Y,v},t}\mathbf{z}_{0,t} & n=0 \\ \mathbf{W}^{(h)}_{\mathrm{S,v},t}\mathbf{z}_{n,t} & n=1,\ldots,K_m, \end{cases} \tag{19}$$

where learnable projection matrices for query, key, and value are $\{\mathbf{W}^{(h)}_{\mathrm{Y,q},t}, \mathbf{W}^{(h)}_{\mathrm{Y,k},t}, \mathbf{W}^{(h)}_{\mathrm{S,k},t}, \mathbf{W}^{(h)}_{\mathrm{Y,v},t}, \mathbf{W}^{(h)}_{\mathrm{S,v},t}\} \in \mathbb{R}^{d'\times d}$ and $\{\mathbf{q}^{(h)}_{0,t}, \mathbf{k}^{(h)}_{n,t}, \mathbf{v}^{(h)}_{n,t}\} \in \mathbb{R}^{d'}$.

For simplicity, we concatenate $\mathbf{k}^{(h)}_{n,t}$ and $\mathbf{v}^{(h)}_{n,t}$ to obtain $\mathbf{K}^{(h)}_t = [\mathbf{k}^{(h)}_{0,t},\ldots,\mathbf{k}^{(h)}_{K_m,t}]$ and $\mathbf{V}^{(h)}_t = [\mathbf{v}^{(h)}_{0,t},\ldots,\mathbf{v}^{(h)}_{K_m,t}]$. Then, we compute the scaled dot-product attention between the query $\mathbf{q}^{(h)}_{0,t}$ and all keys $\{\mathbf{k}^{(h)}_{n,t}\}_{n=0}^{K_m}$. The scaled attention score vector $\mathbf{e}^{(h)}_{0,t} \in \mathbb{R}^{K_m+1}$ can be calculated by

$$\mathbf{e}^{(h)}_{0,t} = \frac{\mathbf{K}^{(h)T}_t \mathbf{q}^{(h)}_{0,t}}{\sqrt{d'}}, \tag{20}$$

where $\mathbf{K}^{(h)}_t \in \mathbb{R}^{d'\times(K_m+1)}$ and $\sqrt{d'}$ is the scaling factor. The purpose of scaling is to ensure that the value of dot product does not become too large due to the large dimension $d'$, thereby avoiding numerical issues such as gradient explosions.

Inspired by RealFormer [32], the residual attention score is applied in our NN. The benefits of it are similar to those of the residual connection proposed in [33], including reducing training difficulty and stabilizing the training in deep NNs. With the residual attention score, each sparse attention transformer encoder takes the raw attention score $\mathbf{e}^{(h)}_{\mathrm{res},t}$ from the previous encoder and adds $\mathbf{e}^{(h)}_{0,t}$ to it to obtain the final attention score. The obtained final attention score will be viewed as the raw attention score $\mathbf{e}^{(h)}_{\mathrm{res},t+1}$ in the next $(t+1)$-th encoder. Then, the final attention score is sent to the softmax function for normalization, which outputs the normalized attention score vector $\mathbf{e}^{(h)}_t$. The above operations can be summarized as

$$\mathbf{e}^{(h)}_t = \mathrm{softmax}(\mathbf{e}^{(h)}_{0,t} + \mathbf{e}^{(h)}_{\mathrm{res},t}), \tag{21}$$

$$\mathbf{e}^{(h)}_{\mathrm{res},t+1} = \mathbf{e}^{(h)}_{0,t} + \mathbf{e}^{(h)}_{\mathrm{res},t}, \tag{22}$$

where $\mathrm{softmax}(\mathbf{x}) = \left[\frac{\exp(x_1)}{\sum\exp(x_i)},\ldots,\frac{\exp(x_K)}{\sum\exp(x_i)}\right]^T$. Multiplying the normalized attention score vector $\mathbf{e}^{(h)}_t$ by value $\mathbf{V}^{(h)}_t$, the output of the $h$-th attention head for $\mathbf{z}_{0,t}$ is obtained:

$$\tilde{\mathbf{z}}^{(h)}_{0,t} = \mathbf{V}^{(h)}_t \mathbf{e}^{(h)}_t, \tag{23}$$

where $\mathbf{V}^{(h)}_t \in \mathbb{R}^{d'\times(K_m+1)}$ and $\mathbf{e}^{(h)}_t \in \mathbb{R}^{K_m+1}$. Since each user's pilot sequence is only correlated with the received signal and independent of other users' pilot sequences, the attention score vector of each user's pilot sequence are all 0 except the score between this user's pilot sequence and the received signal. After normalizing the attention score vector, the output of the $h$-th attention head for $\{\mathbf{z}_{n,t}\}_{n=1}^{K_m}$ is

$$\tilde{\mathbf{z}}^{(h)}_{n,t} = \mathbf{v}^{(h)}_{0,t} \quad n=1,\ldots,K_m. \tag{24}$$

The structure of sparse attention mechanism in the $h$-th attention head is summarily depicted in Fig. 6, which intuitively demonstrates the computation process of sparse attention.

With the output of each attention head, the final output of the sparse multi-head attention module can be calculated by weighted summation. As before, distinct projection matrices $\mathbf{W}^{(h)}_{\mathrm{Y},t}$ and $\mathbf{W}^{(h)}_{\mathrm{S},t}$ are utilized to project $\tilde{\mathbf{z}}^{(h)}_{0,t}$ and $\{\tilde{\mathbf{z}}^{(h)}_{n,t}\}_{n=1}^{K_m}$ back to $d$-dimensional vectors, respectively. As shown in Fig. 5, with learnable matrices $\{\mathbf{W}^{(h)}_{\mathrm{Y},t}, \mathbf{W}^{(h)}_{\mathrm{S},t}\} \in \mathbb{R}^{d\times d'}$, the output of the sparse multi-head attention module is

$$\tilde{\mathbf{z}}_{n,t} = \begin{cases} \sum_{h=1}^{H} \mathbf{W}^{(h)}_{\mathrm{Y},t}\tilde{\mathbf{z}}^{(h)}_{0,t} & n=0 \\ \sum_{h=1}^{H} \mathbf{W}^{(h)}_{\mathrm{S},t}\tilde{\mathbf{z}}^{(h)}_{n,t} & n=1,\ldots,K_m. \end{cases} \tag{25}$$

The Add & Norm module means the residual connection and batch normalization, which make it easier to train deep

NNs. Due to the residual connection which directly passes the input to the output, the NN has the option to learn a transformation close to an identity mapping, rather than learning the entire mapping from scratch, thereby facilitating the training of deep NNs. The batch normalization is employed to normalize the output of each layer, giving it a zero mean and unit variance, which facilitates better gradient propagation between different layers and accelerates the training.

The result of the residual connection is expressed as

$$\mathbf{z}'_{n,t} = \tilde{\mathbf{z}}_{n,t} + \mathbf{z}_{n,t}, \tag{26}$$

and the following operation is batch normalization. The output of Add & Norm module is $\hat{\mathbf{z}}_{n,t} = \mathrm{BN}(\mathbf{z}'_{n,t})$. Next, we will explain the BN operation. Let $\{\mathbf{z}'_{0,t,i}, \ldots, \mathbf{z}'_{K_m,t,i}\}_{i=1}^{I}$ denote a batch of training samples with $I$ samples, and the $j$-th element of $\hat{\mathbf{z}}_{n,t} \in \mathbb{R}^d$ is

$$\hat{\mathbf{z}}_{n,t}(j) = \begin{cases} \dfrac{\mathbf{z}'_{n,t}(j) - \mu_{\mathrm{Y},j}}{\sqrt{\delta_{\mathrm{Y},j}}} & n = 0 \\ \dfrac{\mathbf{z}'_{n,t}(j) - \mu_{\mathrm{S},j}}{\sqrt{\delta_{\mathrm{S},j}}} & n = 1, \ldots, K_m, \end{cases} \tag{27}$$

where $\{\mu_{\mathrm{Y},j}, \mu_{\mathrm{S},j}\}$ represents the mean and $\{\delta_{\mathrm{Y},j}, \delta_{\mathrm{S},j}\}$ denotes the variance. They can be calculated as follows:

$$\mu_{\mathrm{Y},j} = \frac{1}{I}\sum_{i=1}^{I} \mathbf{z}'_{0,t,i}(j), \ \delta_{\mathrm{Y},j} = \frac{1}{I}\sum_{i=1}^{I} (\mathbf{z}'_{0,t,i}(j) - \mu_{\mathrm{Y},j})^2, \tag{28}$$

$$\mu_{\mathrm{S},j} = \frac{1}{IK_m}\sum_{i=1}^{I}\sum_{n=1}^{K_m} \mathbf{z}'_{n,t,i}(j), \tag{29}$$

$$\delta_{\mathrm{S},j} = \frac{1}{IK_m}\sum_{i=1}^{I}\sum_{n=1}^{K_m} (\mathbf{z}'_{n,t,i}(j) - \mu_{\mathrm{S},j})^2. \tag{30}$$

Since the sparse multi-head attention module only involves linear representations, the feed forward module containing the nonlinear activation function is added to introduce the nonlinear mapping capability, enabling NNs to learn more complex representations. Specifically, the feed forward module consists of a two-layer MLP, where the first layer utilizes the ReLU nonlinear activation function, and the second layer has no nonlinear activation function. The output is $\bar{\mathbf{z}}_{n,t} = \mathrm{FF}(\hat{\mathbf{z}}_{n,t})$ and $\bar{\mathbf{z}}_{n,t} \in \mathbb{R}^d$ can be computed by

$$\bar{\mathbf{z}}_{n,t} = \begin{cases} \mathbf{W}^{\mathrm{f2}}_{\mathrm{Y},t}\mathrm{ReLU}(\mathbf{W}^{\mathrm{f1}}_{\mathrm{Y},t}\hat{\mathbf{z}}_{0,t} + \mathbf{b}^{\mathrm{f1}}_{\mathrm{Y},t}) + \mathbf{b}^{\mathrm{f2}}_{\mathrm{Y},t} & n = 0 \\ \mathbf{W}^{\mathrm{f2}}_{\mathrm{S},t}\mathrm{ReLU}(\mathbf{W}^{\mathrm{f1}}_{\mathrm{S},t}\hat{\mathbf{z}}_{n,t} + \mathbf{b}^{\mathrm{f1}}_{\mathrm{S},t}) + \mathbf{b}^{\mathrm{f2}}_{\mathrm{S},t} & n = 1, \ldots, K_m, \end{cases} \tag{31}$$

where $\{\mathbf{W}^{\mathrm{f1}}_{\mathrm{Y},t}, \mathbf{W}^{\mathrm{f1}}_{\mathrm{S},t}\} \in \mathbb{R}^{d_{\mathrm{f}} \times d}$ and $\{\mathbf{b}^{\mathrm{f1}}_{\mathrm{Y},t}, \mathbf{b}^{\mathrm{f1}}_{\mathrm{S},t}\} \in \mathbb{R}^{d_{\mathrm{f}}}$ are weight matrix and bias vector of the first layer respectively while $\{\mathbf{W}^{\mathrm{f2}}_{\mathrm{Y},t}, \mathbf{W}^{\mathrm{f2}}_{\mathrm{S},t}\} \in \mathbb{R}^{d \times d_{\mathrm{f}}}$ and $\{\mathbf{b}^{\mathrm{f2}}_{\mathrm{Y},t}, \mathbf{b}^{\mathrm{f2}}_{\mathrm{S},t}\} \in \mathbb{R}^{d}$ are weight matrix and bias vector of the second layer, respectively. The dimension of the first layer's output is denoted as $d_{\mathrm{f}}$, and $\mathrm{ReLU}(x) = \max(0, x)$.

After the feed forward module, the residual connection and batch normalization are also applied and the final output of the $t$-th sparse attention transformer encoder is $\mathbf{z}_{n,t+1} = \mathrm{BN}(\bar{\mathbf{z}}_{n,t} + \hat{\mathbf{z}}_{n,t})$. The final output of sparse attention transformer block is $\mathbf{z}^{\mathrm{sa}}_n = \mathbf{z}_{n,T+1}, n \in \{0, \ldots, K_m\}$, where $\mathbf{z}_{n,T+1}$ is the output of the $T$-th sparse attention transformer encoder.

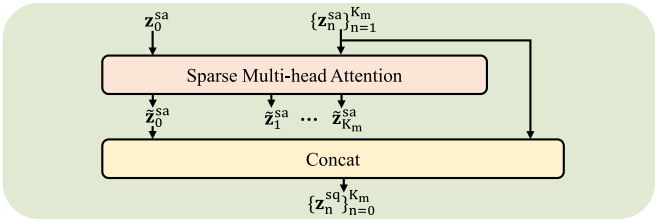



Fig. 7. The detailed structure of the single query attention module.

*3) Single Query Attention:* Here, the high-dimensional representation of the received signal and each user's pilot sequence has been obtained via the sparse attention transformer block module and the following single query attention module will output more complex and abstract representation of the received signal, preparing for further exploring direct correlation between the received signal and each user's pilot sequence in the correlation detection module.

The details of the single query attention module is shown in Fig. 7, where the main component is the sparse multi-head attention introduced before. The distinction is that only the first vector of the output $\tilde{\mathbf{z}}^{\mathrm{sa}}_0$ is useful and other vectors of the output are ignored. In other words, only $\mathbf{z}^{\mathrm{sa}}_0$ is considered as a single query. Concatenating $\tilde{\mathbf{z}}^{\mathrm{sa}}_0$ and $\{\mathbf{z}^{\mathrm{sa}}_n\}_{n=1}^{K_m}$, we can attain the final output $\{\mathbf{z}^{\mathrm{sq}}_n\}_{n=0}^{K_m} \in \mathbb{R}^d$. Employing mathematical expressions, the $\{\mathbf{z}^{\mathrm{sq}}_n\}_{n=0}^{K_m}$ can be obtained as follows:

$$\{\tilde{\mathbf{z}}^{\mathrm{sa}}_n\}_{n=0}^{K_m} = \mathrm{SMA}(\{\mathbf{z}^{\mathrm{sa}}_n\}_{n=0}^{K_m}), \tag{32}$$

$$\mathbf{z}^{\mathrm{sq}}_n = \begin{cases} \tilde{\mathbf{z}}^{\mathrm{sa}}_0 & n = 0 \\ \mathbf{z}^{\mathrm{sa}}_n & n = 1, \ldots, K_m, \end{cases} \tag{33}$$

where the SMA function has been introduced before and we refrain from further elaboration here.

*4) Correlation Detection:* The previous single query attention module has already output more complex and abstract representation of the received signal, kept the representation of each user's pilot sequence unchanged and concatenated them to obtain $\{\mathbf{z}^{\mathrm{sq}}_n\}_{n=0}^{K_m}$, which facilitates calculating the correlation score between $\mathbf{z}^{\mathrm{sq}}_0$ and each $\mathbf{z}^{\mathrm{sq}}_n$ $(n = 1, \ldots, K_m)$ in the correlation detection module. The correlation score $a_n$ between the presentation of the received signal $\mathbf{z}^{\mathrm{sq}}_0$ and the presentation of the $n$-th user's pilot sequence $\mathbf{z}^{\mathrm{sq}}_n$ is

$$a_n = \alpha_{\mathrm{c}}\ \mathrm{softsign}\left(\frac{\left(\mathbf{z}^{\mathrm{sq}}_0\right)^T \mathbf{W}_{\mathrm{c}}\mathbf{z}^{\mathrm{sq}}_n}{\sqrt{d}}\right),\ n = 1, \ldots, K_m, \tag{34}$$

where $\mathbf{W}_{\mathrm{c}} \in \mathbb{R}^{d \times d}$ is a learnable parameter and the element in the $i$-th row and $j$-th column of $\mathbf{W}_{\mathrm{c}}$ is the weight assigned to the product of the $i$-th element of $\mathbf{z}^{\mathrm{sq}}_0$ and the $j$-th element of $\mathbf{z}^{\mathrm{sq}}_n$. When training the NN, we aim to obtain a reasonable $\mathbf{W}_{\mathrm{c}}$ so that the correlation score $a_n$ accurately reflects the activity probability of the $n$-th user. In addition, $\mathrm{softsign}(x) = x/(1 + |x|)$ maps the value to the range $[-1, 1]$ and $\alpha_{\mathrm{c}}$ is a tuning hyperparameter controlling the value within a reasonable range. The logit vector of AP $m$ is denoted by

$\mathbf{a}_m = [a_1, \ldots, a_{K_m}]$, which is the final output of the NN in AP $m$. In fact, the activity probability vector will be obtained after the logit vector $\mathbf{a}_m$ is activated by the sigmoid function.

*5) Training Procedure of NN in Each AP:* The previous content has presented specific components of the NN in each AP, and the following will introduce the training procedure. During the training procedure, gradients of the NN are calculated with a batch of training samples and the Adam optimizer is used to update NN parameters based on calculated gradients. A learning rate decay strategy is employed, and when the specified epoch number is reached, the learning rate will be multiplied by a decay factor, aiding in accelerating the convergence of training. The output of training procedure is optimized NN parameters of all APs $\{\Theta_1, \ldots, \Theta_M\}$, which will be transmitted to the CPU. The training procedure of the NN in AP $m$ is summarized in Algorithm 2.

**Algorithm 2** Training Procedure of the NN in AP $m$

**Input:** number of epochs $N_{\mathrm{e}}$, batch size $N_{\mathrm{b}}$, learning rate $\eta$, decay epoch $N_{\mathrm{d}}$ and decay factor $\gamma$
**for** epoch $= 1 : N_{\mathrm{e}}$ **do**
  Generate a batch of $N_{\mathrm{b}}$ training samples
  Calculate gradients using loss function (6), (7), or (8)
  Update NN parameters using Adam optimizer with $\eta$
  **if** epoch = $N_{\mathrm{d}}$ **then**
    $\eta \leftarrow \gamma\eta$
  **end if**
**end for**
**Output:** optimized NN parameters $\Theta_m$

### B. Fusion in CPU Based on Transfer Learning

Due to the individual bias of each AP caused by different channel characteristics, a fusion module is required to integrate judgments on the same user from distinct APs to obtain more accurate final result. With the existence of fronthaul links in cell-free systems, the CPU can receive user activity state judgments from each AP so the fusion module is positioned in the CPU.

*1) Introduction to Transfer Learning:* Transfer learning is a machine learning technique that leverages knowledge learned from one task to improve the performance on a new task. The core idea of transfer learning involves training a model initially on a source domain, and subsequently transferring the model or its components to a target domain. In addition, parameters of the transferred NN processing original task are usually frozen and only parameters of newly added components in the NN processing new task are fine-tuned, which reduces the training burden.

In our paper, the source domain corresponds to the task where each AP trains its NN to detect activity states of users under its responsibility, while the target domain corresponds to the task where CPU aims to detect activity states of all users. Inspired by the transfer learning, each AP sends its trained NN parameters to the CPU and the UADFormer is obtained by combining the fusion module with transferred NNs of all APs. As NNs of all AP have already been trained, there is no need to retrain the entire NN in the CPU from scratch and we only have to train the fusion module, thereby avoiding unnecessary waste of computational resources.

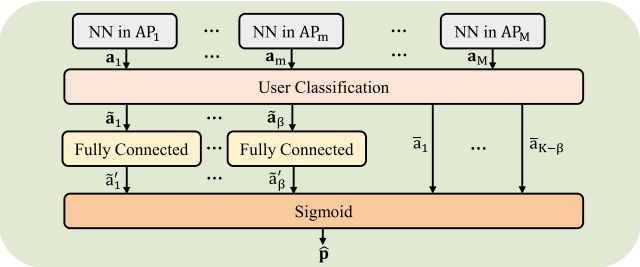



Fig. 8. The detailed structure of the fusion module.

*2) Specific Implementation of Fusion:* The detailed structure of the fusion module is illustrated in Fig. 8 and each component will be introduced below.

With parameters of trained NNs in APs, we can acquire $\{\mathbf{a}_m\}_{m=1}^{M}$ by inputting training samples into trained NNs of APs with frozen parameters. In the user classification module, elements of $\{\mathbf{a}_m\}_{m=1}^{M}$ will be recombined, and activity judgments for the same user will be aggregated, resulting in $\{\tilde{\mathbf{a}}_n\}_{n=1}^{\beta}$ and $\{\bar{a}_n\}_{n=1}^{K-\beta}$. $\{\tilde{\mathbf{a}}_n\}_{n=1}^{\beta}$ represents the logit vector of users detected by multiple APs where $\beta$ represents the number of such users, and $\{\bar{a}_n\}_{n=1}^{K-\beta}$ denotes the logit value of users detected by a single AP.

After the user classification module, $\{\bar{a}_n\}_{n=1}^{K-\beta}$ is directly fed into the sigmoid module while $\{\tilde{\mathbf{a}}_n\}_{n=1}^{\beta}$ needs to pass through a fully connected layer. For $\beta$ users detected by multiple users, we design an individual fully connected layer with non-shared parameters for each user, leading to a total of $\beta$ fully connected layers. The fully connected layer consists of a two-layer MLP with a $d_{\mathrm{fc}}$-dimensional hidden layer employing ReLU activation function, and outputs a scalar. Assuming $\tilde{\mathbf{a}}_n$ is a $d_n$-dimensional vector, the output of the $n$-th fully connected layer is

$$\tilde{a}'_n = (\mathbf{w}_n^{\mathrm{fc}})^T \left(\mathrm{ReLU}\left(\mathbf{W}_n^{\mathrm{fc}}\tilde{\mathbf{a}}_n + \mathbf{b}_n^{\mathrm{fc}}\right)\right) + b_n^{\mathrm{fc}}, \tag{35}$$

where $\mathbf{W}_n^{\mathrm{fc}} \in \mathbb{R}^{d_{\mathrm{fc}} \times d_n}$, $\mathbf{b}_n^{\mathrm{fc}} \in \mathbb{R}^{d_{\mathrm{fc}}}$, $\mathbf{w}_n^{\mathrm{fc}} \in \mathbb{R}^{d_{\mathrm{fc}}}$, $b_n^{\mathrm{fc}} \in \mathbb{R}$ are learnable parameters of the $n$-th fully connected layer. Since the activity probability is between 0 and 1, the sigmoid module is used to map the value to the range $[0, 1]$ and $\mathrm{sigmoid}(x) = 1/(1+e^{-x})$. Feeding $\{\tilde{a}'_n\}_{n=1}^{\beta}$ and $\{\bar{a}_n\}_{n=1}^{K-\beta}$ into the sigmoid module, we can attain the activity probability vector of all users $\hat{\mathbf{p}}$.

*3) Training Procedure of Fusion in CPU:* Before training the fusion module in CPU, CPU has received optimized NN parameters of all APs $\{\Theta_1, \ldots, \Theta_M\}$. These parameters will be set non-trainable when training the fusion module, which means that only parameters of the fusion module denoted as $\Theta_0$ need to be optimized. It is worth noting that in the fusion module, only fully connected layers have learnable parameters and they do not affect each other, meaning that each fully connected layer can be trained independently. In fact, training fully connected layers are similar to those in APs introduced before, and we will not elaborate further here.

### C. Complexity Analysis

In practical systems, NNs are pre-trained offline, so when analyzing computational complexity, we only consider the complexity of online prediction, neglecting the complexity

TABLE I
COMPUTATIONAL COMPLEXITY OF EACH MODULE

| Module | FLOPs | Complexity order | Module | FLOPs | Complexity order |
|---|---|---|---|---|---|
| Preprocessing | $2L^2(3N-1)$ | $\mathcal{O}(L^2N)$ | Linear projection | $f_{\text{LP}}(m) = (4dL^2-d)+ K_m(4dL-d)$ | $\mathcal{O}(dL^2)+ \mathcal{O}(K_m dL)$ |
| Sparse multi-head attention | $f_{\text{SMA}}(m) = H\left[(2K_m+3)d'(2d-1) +(K_m+1)(2d'-1)+d'(2K_m+1)\right] +(K_m+1)d(2Hd'-1)$ | $\mathcal{O}(HK_m d' d)$ | Feed forward | $f_{\text{FF}}(m) = 4(K_m+1)d_{\text{f}}d$ | $\mathcal{O}(K_m d_{\text{f}} d)$ |
| Sparse attention transformer block | $f_{\text{SATB}}(m) = T\left(f_{\text{SMA}}(m)+f_{\text{FF}}(m)\right)$ | $\mathcal{O}(THK_m d' d)+ \mathcal{O}(TK_m d_{\text{f}} d)$ | Single query attention | $f_{\text{SQA}}(m) = f_{\text{SMA}}(m)$ | $\mathcal{O}(HK_m d' d)$ |
| Correlation detection | $f_{\text{CD}}(m) = (K_m+d)(2d-1)$ | $\mathcal{O}(K_m d)+\mathcal{O}(d^2)$ | Fusion | $\sum_{n=1}^{\beta} 2d_{\text{fc}}(d_n+1)$ | $\mathcal{O}(\beta d_{\text{fc}} d_n)$ |

of offline training. The UADFormer is based on NNs in $M$ APs and a CPU, so the overall system complexity is determined by these two components. In the DL, floating point operations (FLOPs) is usually used to measure the complexity of NNs, which is also adopted in this paper. As NNs primarily involve matrix multiplication of weight matrices with feature representation vectors followed by addition of bias vectors, FLOPs refers to the count of multiplication and addition operations involving weight matrices.

The FLOPs for the multiplication of matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ and matrix $\mathbf{B} \in \mathbb{R}^{n\times p}$ is $mnp+m(n-1)p$, which consists of $mnp$ multiplication operations and $m(n-1)p$ addition operations, and the complexity order is $\mathcal{O}(mnp)$. The complexity of each module is shown in Table I, and total complexity comparison between standard transformer and UADFormer is illustrated in Table II, where FLOPs of standard transformer block is $f_{\text{STB}}(m) = T\left(f_{\text{MA}}(m)+f_{\text{FF}}(m)\right)$ and FLOPs of multi-head attention $f_{\text{MA}}(m)$ is shown in (36), as shown at the bottom of the next page. From these two tables, it can be seen that the total complexity order of UADFormer is $\mathcal{O}(MTHK_m d' d) + \mathcal{O}(MTK_m d_{\text{f}} d)$, which is mainly determined by the sparse multi-head attention module. Moreover, numerical FLOPs of UADFormer decreases by $53\%$ compared with that of standard transformer, demonstrating our modification to the standard multi-head attention mechanism can effectively reduce the computational complexity.

## VI. SIMULATION RESULTS

### A. Simulation Settings

Here, we consider a uplink cell-free massive MIMO system with $K = 200$ users uniformly distributed in a square area of 100m $\times$ 100m and $M = 20$ APs are randomly distributed in this area. Similar to [25], the training and testing samples are generated as follows. Each element of the pilot matrix $\mathbf{S}$ is generated obeying i.i.d. $\mathcal{CN}(0,1)$. As in some works about cell-free massive MIMO systems such as [12] and [34], the large-scale fading coefficient $\lambda_{m,k}$ is generated as follows:

$$\lambda_{m,k} = 10^{\frac{\text{PL}_{m,k}}{10}} \cdot 10^{\frac{\text{SH}_{m,k}}{10}}, \tag{37}$$

where $10^{\frac{\text{PL}_{m,k}}{10}}$ denotes the path loss and $10^{\frac{\text{SH}_{m,k}}{10}}$ represents the shadowing effect with $\text{SH}_{m,k} \in \mathcal{N}(0, 8^2)$ (in dB). $\text{PL}_{m,k}$ (in dB) is given by

$$\text{PL}_{m,k} = \begin{cases} -C - 35\log_{10}(d_{m,k}), \\ \quad \text{if } d_{m,k} > d_1 \\ -C - 15\log_{10}(d_1) - 20\log_{10}(d_{m,k}), \\ \quad \text{if } d_0 < d_{m,k} \le d_1 \\ -C - 15\log_{10}(d_1) - 20\log_{10}(d_0), \\ \quad \text{if } d_{m,k} \le d_0, \end{cases} \tag{38}$$

where $d_{m,k}$ denotes the distance in meters between AP $m$ and the user $k$, and $C$ is a constant depending on the carrier frequency, the user and AP heights. Here, in accordance with parameter settings in [35], we set $C = 35.7$ dB, $d_0 = 10$ m, and $d_1 = 50$ m. Each element of small-scale fading vectors is also generated obeying i.i.d. $\mathcal{CN}(0,1)$. The transmission power of each user is 20 dBm and the background Gaussian noise at each AP is $-100$ dBm. Active users are generated by Bernoulli sampling and the received signal of each AP is generated according to (2).

Considering the cost of collecting data in reality, it is nearly impossible to generate new training samples at each training epoch since this requires a very large amount of data. Thereby, as in [29] and [36], we generate, save and repeatedly use 110000 pieces of samples for NN training and testing. Then, we distribute the training data set and test data set by 10:1, which resembles that in [29]. Then, all users are assigned to different APs for detection using our proposed user grouping algorithm, and UADFormer can be trained utilizing BCE, WBCE, or ZLPR loss function.

The hyper-parameters of UADFormer are divided into two parts, the NN hyper-parameters in APs and in the CPU. Referring to [25], we choose appropriate hyper-parameter values. The value of each hyper-parameter of NNs in APs is illustrated in Table III, and hyper-parameters of the NN in the CPU are generally the same as those of APs except the number of training epochs and decay epoch, which are 15 and 10, respectively. In addition, the hidden size of the fusion module is $d_{\text{fc}} = 256$, and the predefined threshold $\tau$ is usually set as 0.5.

The direct evaluation metric is the detection error rate (ER). Assuming $\hat{\mathbf{x}}$ and $\mathbf{x}$ respectively represent the predicted and ground truth activity state vector with the length of $K$, ER is

TABLE II
TOTAL COMPLEXITY COMPARISON ($K = 200, M = 20, L = 10, N = 16, d = 128, d' = 32, d_{\mathrm{f}} = 512, d_{\mathrm{fc}} = 256, H = 8, T = 4$)

| DL model | Total FLOPs | Total complexity order | Numerical FLOPs |
|---|---|---|---|
| Standard transformer | $\sum_{m=1}^{M}\left(2L^2(3N-1)+f_{\mathrm{LP}}(m)+f_{\mathrm{STB}}(m)+f_{\mathrm{SQA}}(m)+f_{\mathrm{CD}}(m)\right)+\sum_{n=1}^{\beta}2d_{\mathrm{fc}}(d_n+1)$ | $\mathcal{O}(MTHK_m d' d)+\mathcal{O}(MTHd' K_m^2)+\mathcal{O}(MTK_m d_{\mathrm{f}} d)$ | $5.07\times10^8$ |
| UADFormer | $\sum_{m=1}^{M}\left(2L^2(3N-1)+f_{\mathrm{LP}}(m)+f_{\mathrm{SATB}}(m)+f_{\mathrm{SQA}}(m)+f_{\mathrm{CD}}(m)\right)+\sum_{n=1}^{\beta}2d_{\mathrm{fc}}(d_n+1)$ | $\mathcal{O}(MTHK_m d' d)+\mathcal{O}(MTK_m d_{\mathrm{f}} d)$ | $4.57\times10^8$ |

TABLE III
HYPER-PARAMETERS OF NNS IN APS

| Hyper-parameter | Value | Hyper-parameter | Value | Hyper-parameter | Value |
|---|---|---|---|---|---|
| Number of epochs $N_{\mathrm{e}}$ | 100 | Batch size $N_{\mathrm{b}}$ | 200 | Learning rate $\eta$ | $2\times10^{-4}$ |
| Decay epoch $N_{\mathrm{d}}$ | 80 | Decay factor $\gamma$ | 0.1 | Number of attention heads $H$ | 8 |
| Number of transformer encoders $T$ | 4 | Linear projection size $d$ | 128 | Dimension of attention space $d'$ | 32 |
| Hidden size of FF module $d_{\mathrm{f}}$ | 512 | Tuning parameter $\alpha_{\mathrm{p}}$ | 500 | Tuning parameter $\alpha_{\mathrm{c}}$ | 10 |

defined as

$$\mathrm{ER} = 1 - \frac{\hat{\mathbf{x}}^T\mathbf{x} + (1-\hat{\mathbf{x}})^T(1-\mathbf{x})}{K}. \tag{39}$$

Considering the imbalance of UAD, as in [25] and [37], the probability of missed detection (PM) and the probability of false alarm (PF) are also employed to evaluate the UAD performance, which are defined as follows:

$$\mathrm{PM} = \frac{\mathbf{x}^T(\mathbf{1}-\hat{\mathbf{x}})}{\|\mathbf{x}\|_0}, \quad \mathrm{PF} = \frac{(\mathbf{1}-\mathbf{x})^T\hat{\mathbf{x}}}{\|\mathbf{1}-\mathbf{x}\|_0}, \tag{40}$$

where $\mathbf{1}$ is a vector with all elements being 1 and $\|\cdot\|_0$ denotes the 0 norm of vector. In addition, unless otherwise specified, in the following simulation, the user activity ratio is $p_{\mathrm{a}} = 0.1$, the number of antennas of each AP is set as $N = 16$, and the length of pilot sequences is $L = 10$.

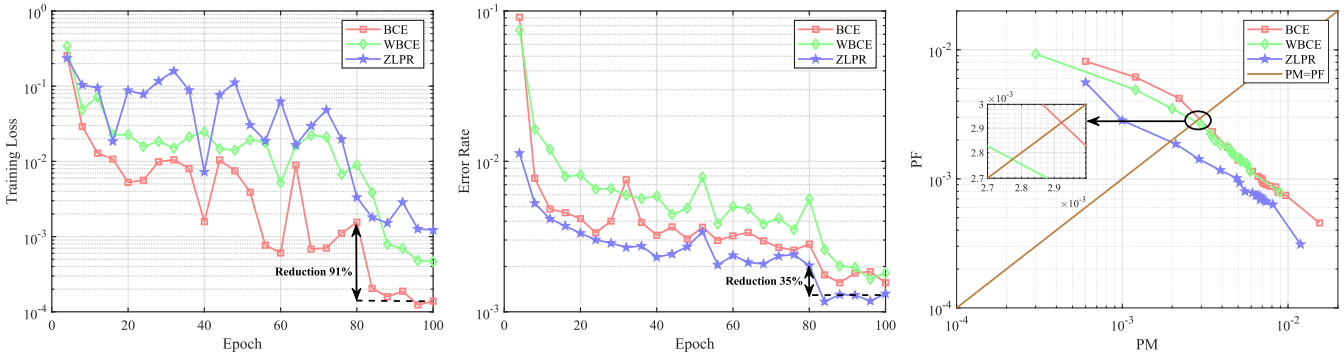
### B. Performance Evaluation

Firstly, different evaluation metrics for UADFormer using various loss functions versus training epochs during the training procedure in APs are illustrated in Fig. 9. From Fig. 9a and Fig. 9b, It can be seen that the training loss and testing ER generally decrease with an increase of the training epoch and at decay epoch $N_{\mathrm{d}} = 80$, the training loss and testing ER decrease rapidly, which indicates the effectiveness of the learning rate decay strategy. Moreover, observing these three curves in Fig. 9b, the testing ER using ZLPR loss function is the lowest among the three, which indicates the benefit of ZLPR. The ER employing BCE is lower than that utilizing WBCE at the last training epoch, which seems unreasonable at first glance. To address this, the PM-PF curves of these three loss functions with different predefined thresholds, which illustrate the classification balance, are shown in Fig. 9c. The intersections of the three curves with the PM = PF line should be given special attention, which reflect the accuracy of UAD in balancing active and inactive users. In Fig. 9c, PM and PF of ZLPR's intersection are lowest, once again proving its superiority. Furthermore, PM and PF of WBCE's intersection are lower than those of BCE's intersection, which demonstrates that UADFormer using WBCE has better classification balance.

Fig. 10 presents the training loss and ER versus epoch when training the fusion module in CPU. Since Fig. 9 has illustrated advantages of ZLPR, ZLPR is utilized to train the fusion module and its training loss and ER are termed as TL-Fusion and ER-Fusion, respectively. In addition, unless otherwise specified, ZLPR is used as the training loss function for the subsequent simulations. To highlight the necessity of fusion, ER calculated by taking the average of correlation scores provided by multiple APs, termed as ER-AVG, is also depicted in Fig. 10 for comparison. It can be seen that the training loss and ER generally decrease as the number of epochs increases. More importantly, after 8 epochs, ER-Fusion is lower than ER-AVG, and at the 15th epoch, ER-Fusion decreases by 27% compared with ER-AVG, indicating that the fusion module can effectively reduce the ER.

Next, PM-PF curves of UADFormer and six other schemes are shown in Fig. 11. UADFormer is termed as UF and standard transformer without sparse attention mechanism is termed as ST. The performance of MLP, consisting of 6 hidden layers with batch normalization and ReLU activation, is also illustrated in Fig. 11. As for traditional schemes, the CS based approach is not adopted since instantaneous CSI needs to be estimated, which differs significantly from UADFormer. The covariance based method in [19], termed as COV, is adpoted as instantaneous CSI is not needed. The performance of traditional DL algorithm used to solve

$$f_{\mathrm{MA}}(m) = H\left[3(K_m+1)d'(2d-1)+(K_m+1)^2(2d'-1)+d'(K_m+1)(2K_m+1)\right]+(K_m+1)d(2Hd'-1). \tag{36}$$




(a) Training loss versus epoch in APs. (b) Error rate versus epoch in APs. (c) PM-PF with different predefined thresholds.

Fig. 9. Different evaluation metrics for UADFormer using various loss functions versus training epoch during the training procedure in APs.

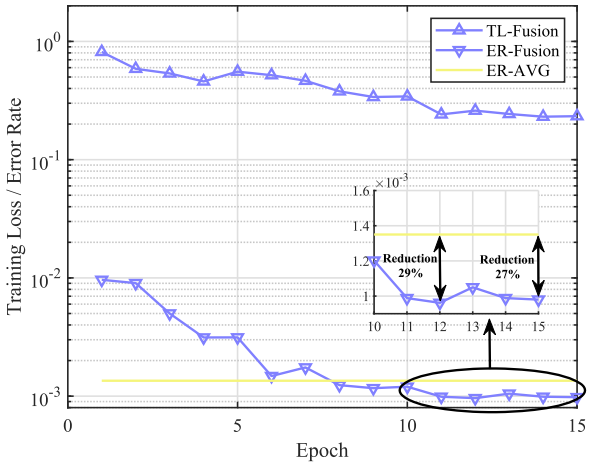



Fig. 10. Training loss and error rate versus epoch when training the fusion module in the CPU.

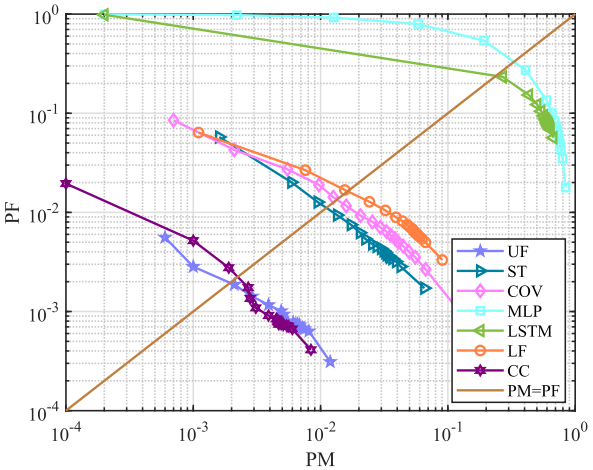



Fig. 11. PM-PF performance of different schemes.

sequential problems such as long short-term memory (LSTM) proposed in [38], which is termed as LSTM, is also illustrated in Fig. 11. In addition, in order to demonstrate the benefits of our proposed UADFormer, we compare the performance of low-complexity transformer-based scheme Linformer proposed in [39] termed as LF, with that of UADFormer in Fig. 11. Moreover, the performance of heterogeneous transformer proposed in [25] used in centralized cellular system, termed as CC, is depicted in Fig. 11 to show performance

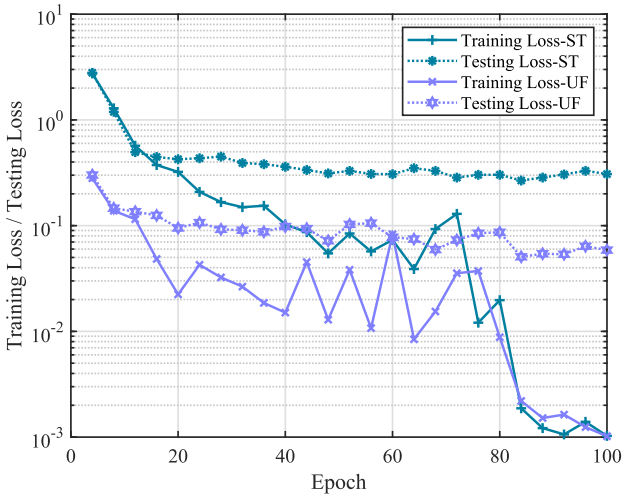



Fig. 12. Training loss and testing loss of standard transformer and UADFormer versus epoch at APs.

comparison between centralized cellular system and cell-free system. It can be seen from Fig. 11 that the intersection point of UADFormer and PM = PF line has lowest PM and PF among all methods, which indicates the advantages of our proposed UADFormer in terms of detection accuracy and classification balance. Additionally, the UAD performance of heterogeneous transformer in the centralized cellular system is close to that of UADFormer in the cell-free system. The performance of low-complexity Linformer is lower than that of standard transformer because it does not specifically design a low-complexity NN structure based on the characteristics of the UAD problem like UADFormer. Moreover, the covariance based method does not perform as well as UADFormer, and the rest two methods, LSTM-based and MLP-based methods, has much lower performance than UADFormer since they lack the NN structure specifically designed for the UAD problem. In general, the simulation results in Fig. 11 highlight the benefits of our proposed UADFormer.

After carefully observing Fig. 11, it is somewhat strange that UADFormer performs better than standard transformer since standard transformer has more learnable parameters and should have higher detection accuracy. Considering only 100000 pieces of samples are used for training, one possible reason is that the overfitting occurs in the training of standard transformer. To verify it, we provide the training loss and

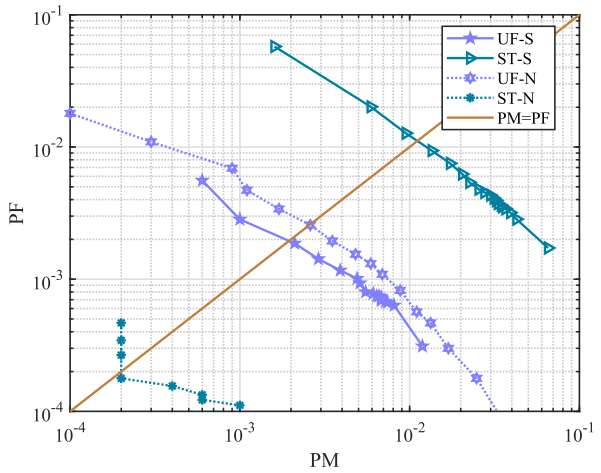



Fig. 13. PM-PF performance of two different data generation methods.

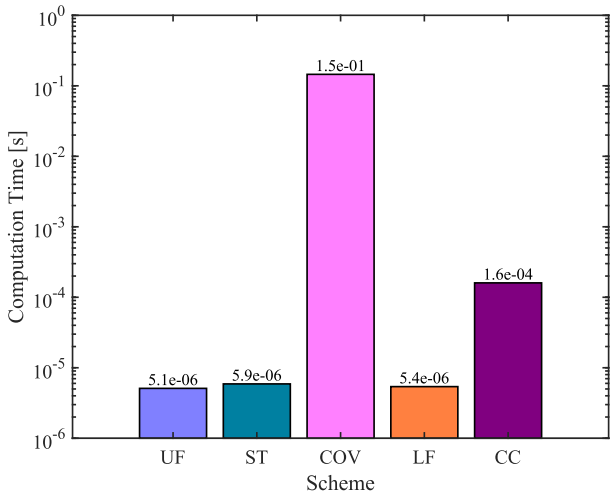



Fig. 14. Computation time of different schemes.

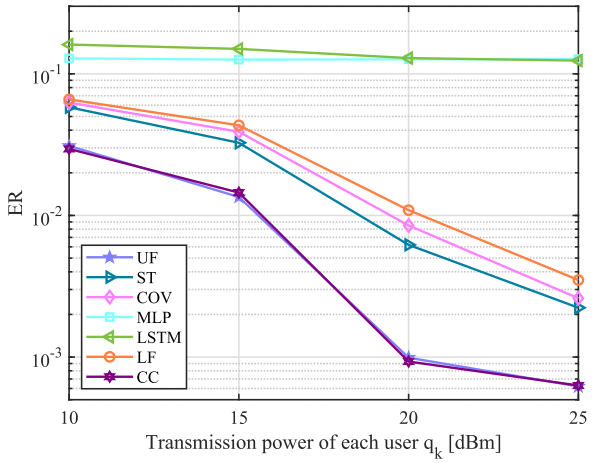



Fig. 15. ER versus transmission power of each user $q_k$.

testing loss of standard transformer and UADFormer in Fig. 12. From Fig. 12, we can see that there is a gap between training loss and testing loss of standard transformer and UADFormer, which indicates that overfitting indeed occurs during training phase of standard transformer and UADFormer. However, the gap of standard transformer is greater than that of UADFormer, which means overfitting of standard transformer is more severe than that of UADFormer, so the performance of UADFormer is better than that of standard transformer in Fig. 11.

As relevant paper [25] generates new data at each training epoch, which can effectively avoid overfitting but requires collecting a very large amount of data, we test the performance of standard transformer and UADFormer using the data generation method in [25]. The results are shown in Fig. 13, where the data generation method that we generate, save and repeatedly use 100000 pieces of samples for NN training is termed as UF-S and ST-S corresponding to UADFormer and standard transformer, and the data generation method in [25] that we generate a very large amount of data and use new data at each training epoch is termed as UF-N and ST-N. It can be seen from Fig. 13 that the intersection point of standard transformer using the data generation method in [25] and PM = PF line has lowest PM and PF among all methods, which demonstrates that if there is enough training data, standard transformer with more learnable parameters will perform better than UADFormer.

From Fig. 12 and Fig. 13, we can conclude that 100000 pieces of training samples are not enough for standard transformer and UADFormer, resulting the overfitting but due to the NN structure designed based on characteristics of the UAD problem, UADFormer performs significantly better than standard transformer and its detection accuracy is acceptable. If a sufficient amount of training data is used, the performance of standard transformer will be better. Considering the cost of collecting data in reality, our proposed UADFormer is a better choice in reality.

To illustrate low computational complexity of our proposed UADFormer intuitively, we compare the computation time for solving one UAD problem of all schemes except LSTM and MLP because of their very poor performance. Here, only COV is implemented on the 13th Intel(R) Core(TM) i9-13900 CPU @ 2.00 GHz while UF, ST, LF and CC are implemented on NVIDIA GeForce RTX 4090. The results are shown in Fig. 14 and it can be seen that the computation time of UADFormer is minimal, which indicates that the computational complexity of UADFormer is lowest, and the computation time of Linformer and standard transformer is close to that of UADFormer. The computation time of heterogeneous transformer for solving the UAD problem in the centralized cellular system is about 30 times that of UADFormer for solving the UAD problem in the cell-free system since all computing tasks are assigned to a single base station for processing in the centralized cellular system. The computation time of covariance based scheme is about 30000 times of that of UADFormer since it requires continuous iteration to obtain the final result.

Subsequently, the performance of various schemes under different parameters of communication systems will be illustrated. In fact, since ER of MLP and LSTM is around 10%, which is too high, they will not be utilized in reality. Thereby, here, we only focus on other schemes to explore the variation of ER with changes in communication system parameters. First, we conduct simulation experiments to obtain ER of different schemes with different transmission power of each user, and results are shown in Fig. 15. From Fig. 15, we can

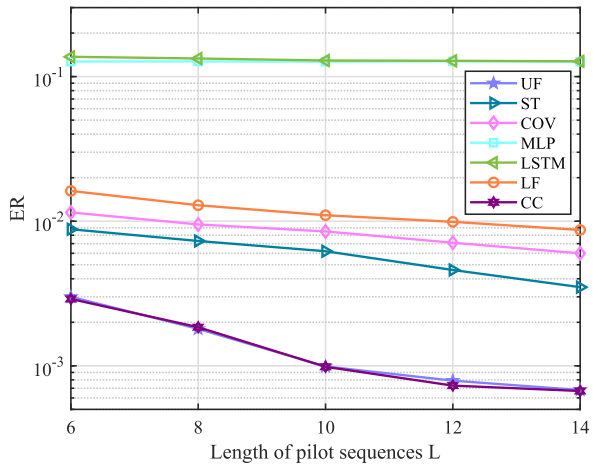



Fig. 16. ER versus length of pilot sequences $L$.

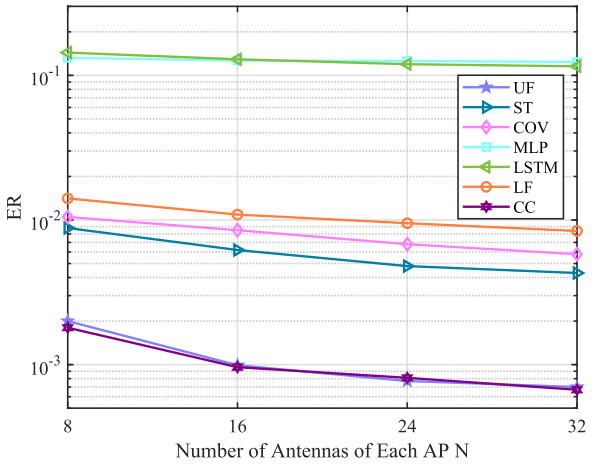



Fig. 17. ER versus number of antennas of each AP $N$.

find that as the transmission power of each user increases, ER of all schemes expect MLP will decline. In addition, our proposed UADFormer has lowest ER among all schemes used in cell-free systems and its performance is close to that of CC used in centralized cellular systems. Then, ER versus length of pilot sequences is depicted in Fig. 16, where the ER of all schemes except MLP declines with the increase of the length of pilot sequences. Fig. 17 illustrates ER versus number of antennas of each AP, and it can be seen from Fig. 17 that as the number of antennas increases, ER of all schemes will decline. Consistent with previous simulation results, the performance of our proposed UADFormer in Fig. 16 and Fig. 17 is lower than that of other schemes used in cell-free systems.

Considering that spatial correlation among antennas might exist, the PM-PF performance of our proposed UADFormer with different correlation coefficients is illustrated in Fig. 18, where $\rho$ denotes the spatial correlation coefficient and spatially uncorrelated fading channel is termed as SUC. As in [40], the correlation matrix is modeled by a exponential model with different correlation coefficients. From Fig. 18, we can see that PM-PF performance declines with the increase of correlation coefficient, which demonstrates that stronger spatial correlation leads to poorer PM-PF performance. However, even when $\rho = 0.7$, which means large spatial correlation, the PM and PF are both less than $0.8\%$ when PM $=$ PF. The PM and

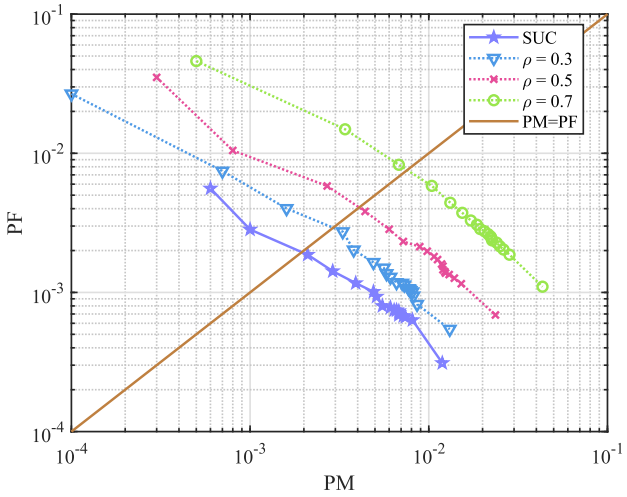



Fig. 18. PM-PF performance of our proposed UADFormer with different correlation coefficients.

TABLE IV

TABLE OF ABLATION STUDY

| Component | Choice | | |
|---|---|---|---|
| Residual attention score | ✓ | | ✓ |
| Sparse attention mechanism | ✓ | ✓ | |
| Standard attention mechanism | | | ✓ |
| ER (%) | **0.09** | 0.12 | 0.62 |
| Number of parameters | **44.24M** | **44.24M** | 46.95M |
| FLOPs | **457M** | **457M** | 507M |

PF are completely acceptable when $\rho = 0.7$, which highlights the robustness of our proposed UADFormer under spatially correlated fading.

### C. Ablation Study

To demonstrate effects of our changes on the NN, we conduct the ablation study and results are shown in Table IV. From Table IV, we can see that the residual attention score can effectively improve the performance of NNs without increasing the number of parameters and FLOPs. In addition, compared with standard attention mechanism, sparse attention mechanism efficiently reduces the number of parameters and FLOPs while improving detection accuracy.

## VII. CONCLUSION

This paper proposed UADFormer, which was a transformer-based deep learning method to solve the user activity detection problem with changeable pilot sequences in the cell-free massive MIMO system. Users were assigned to APs for detection employing our proposed user grouping algorithm based on large-scale fading coefficients. The transformer-based NN was deployed in each access point and it output user activity state by exploring the correlation between received signals and pilot sequences. The fusion NN based on transfer learning was deployed in the central processing unit to determine final activity states of users detected by multiple access points. Moreover, given limited computational and storage resources in access points, we utilized sparse attention mechanism to reduce complexity and the residual attention score to enhance the performance of NNs. Simulation results showed the

$$\begin{aligned}\mathbb{E}\{\mathbf{Y}_m\mathbf{Y}_m^H\} &= \mathbb{E}\left\{\left(\sum_{k=1}^{K} x_k\sqrt{q_k}\sqrt{\lambda_{m,k}}\mathbf{s}_k\mathbf{h}_{m,k}^T + \mathbf{W}_m\right)\left(\sum_{k=1}^{K} x_k\sqrt{q_k}\sqrt{\lambda_{m,k}}\mathbf{h}_{m,k}^*\mathbf{s}_k^H + \mathbf{W}_m^H\right)\right\} \\ &= \mathbb{E}\left\{\sum_{k=1}^{K} x_k^2 q_k\lambda_{m,k}\mathbf{s}_k\mathbf{h}_{m,k}^T\mathbf{h}_{m,k}^*\mathbf{s}_k^H + \mathbf{W}_m\mathbf{W}_m^H\right\} = N\sum_{k=1}^{K} q_k\lambda_{m,k}\mathbf{s}_k\mathbf{s}_k^H\mathbb{E}\{x_k^2\} + \mathbb{E}\left\{\mathbf{W}_m\mathbf{W}_m^H\right\} \\ &= Np_a\sum_{k=1}^{K} q_k\lambda_{m,k}\mathbf{s}_k\mathbf{s}_k^H + N\sigma_m^2\mathbf{I}_L. \end{aligned} \tag{41}$$

superiority of UADFormer in various evaluation metrics compared with other schemes.

## Appendix A
## Method for Obtaining $K_a$

Here, we are committed to estimating the value of activity ratio $p_a$, and $K_a$ can be obtained by calculating $K_a = K \cdot p_a$. We discover that the activity ratio $p_a$ is related to the covariance matrix of the received signal. Assuming that the probability of each user being active is the same and mutually independent, since each element of small-scale fading vectors is generated obeying i.i.d. $\mathcal{CN}(0,1)$, the mathematical expectation of the covariance matrix of received signal at AP $m$, i.e., $\mathbb{E}\{Y_m Y_m^H\}$, can be expressed by (41), as shown at the top of the page.

By collecting a large amount of received signal data at AP $m$, which is also necessary for training neural networks, we can calculate the mean of a large number of samples and employ it as an approximation to the mathematical expectation $\mathbb{E}\{Y_m Y_m^H\}$. Given that in (41), all parameters except $p_a$ are ascertainable, $p_a$ can be derived easily. Then, $K_a$ is estimated using $K_a = K \cdot p_a$.

## Appendix B
## Proof of (11)

To prove the validity of (11), we need to prove that the log-sum-exp operator serves as a smooth approximation to the maximum operator. The first step is to prove that the log-sum-exp operator is smooth, i.e., its continuity and the continuity of its derivatives of various orders within its domain of definition. Apparently, we have

$$1 + \sum_{k=1}^{K} e^{a_k^{(i)}} > 0, \tag{42}$$

so $\log\left(1 + \sum_{k=1}^{K} e^{a_k^{(i)}}\right)$ is continuous. Next, we calculate its first partial derivative and we have

$$\frac{\partial \log\left(1 + \sum_{k=1}^{K} e^{a_k^{(i)}}\right)}{\partial a_k^{(i)}} = e^{a_k^{(i)}} \Big/ \left(1 + \sum_{k=1}^{K} e^{a_k^{(i)}}\right), \tag{43}$$

so the first partial derivative is continuous. Through the same process, the chain rule can be repeatedly applied to prove that any high-order partial derivative is continuous within its domain of definition. Thus, here, we have proved that the log-sum-exp operator is smooth. Afterwards, we will prove from two perspectives that the log-sum-exp operator is an approximation to the maximum operator. Here, $a_{\max}$ is utilized to represent $\max(0, a_1^{(i)}, \ldots, a_K^{(i)})$, i.e., $a_{\max} = \max(0, a_1^{(i)}, \ldots, a_K^{(i)})$, and we have

$$a_{\max} = \log\left(e^{a_{\max}}\right) < \log\left(e^0 + \sum_{k=1}^{K} e^{a_k^{(i)}}\right). \tag{44}$$

In addition, we also have

$$\begin{aligned}&\log\left(e^0 + \sum_{k=1}^{K} e^{a_k^{(i)}}\right) \le \log\left((K+1)\, e^{a_{\max}}\right) \\ &= \log\left(e^{a_{\max}}\right) + \log(K+1) = a_{\max} + \log(K+1). \end{aligned} \tag{45}$$

Based on (44) and (45), we have

$$a_{\max} < \log\left(e^0 + \sum_{k=1}^{K} e^{a_k^{(i)}}\right) \le a_{\max} + \log(K+1), \tag{46}$$

so we can determine that the log-sum-exp operator is an approximation to the maximum operator and their maximum difference is no more than $\log(K+1)$. After proving that the log-sum-exp operator is smooth and an approximation to the maximum operator, the validity of (11) has been proved.

## References


[1] Y. Liu, Y. Deng, M. Elkashlan, A. Nallanathan, and G. K. Karagiannidis, "Optimization of grant-free NOMA with multiple configured-grants for mURLLC," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 4, pp. 1222–1236, Apr. 2022.

[2] S. R. Pokhrel, J. Ding, J. Park, O.-S. Park, and J. Choi, "Towards enabling critical mMTC: A review of URLLC within mMTC," *IEEE Access*, vol. 8, pp. 131796–131813, 2020.

[3] *IMT VisionFramework and Overall Objectives of the Future Development of IMT for 2020 and Beyond*, document M.2083-0, Int. Telecommun. Union, Geneva, Switzerland, Sep. 2015.

[4] *Study on Scenarios and Requirements for Next Generation Access Technologies (release 14)*, document 38.913, 3GPP, Sophia Antipolis, France, 2017.

[5] M. Ke, Z. Gao, Y. Wu, X. Gao, and K.-K. Wong, "Massive access in cell-free massive MIMO-based Internet of Things: Cloud computing and edge computing paradigms," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 3, pp. 756–772, Mar. 2021.

[6] X. You, "6G extreme connectivity via exploring spatiotemporal exchangeability," *Sci. China Inf. Sci.*, vol. 66, no. 3, pp. 94–96, Mar. 2023.

[7] M. B. Shahab, R. Abbas, M. Shirvanimoghaddam, and S. J. Johnson, "Grant-free non-orthogonal multiple access for IoT: A survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 1805–1838, 3rd Quart., 2020.

[8] K. Senel and E. G. Larsson, "Grant-free massive MTC-enabled massive MIMO: A compressive sensing approach," *IEEE Trans. Commun.*, vol. 66, no. 12, pp. 6164–6175, Dec. 2018.

[9] W. Kim, Y. Ahn, and B. Shim, "Deep neural network-based active user detection for grant-free NOMA systems," *IEEE Trans. Commun.*, vol. 68, no. 4, pp. 2143–2155, Apr. 2020.

[10] L. Liu and W. Yu, "Massive connectivity with massive MIMO—Part I: Device activity detection and channel estimation," *IEEE Trans. Signal Process.*, vol. 66, no. 11, pp. 2933–2946, Jun. 2018.

[11] M. Guo and M. C. Gursoy, "Distributed sparse activity detection in cell-free massive MIMO systems," in *Proc. IEEE Global Conf. Signal Inf. Process. (GlobalSIP)*, Nov. 2019, pp. 1–5.

[12] H. Q. Ngo, A. Ashikhmin, H. Yang, E. G. Larsson, and T. L. Marzetta, "Cell-free massive MIMO versus small cells," *IEEE Trans. Wireless Commun.*, vol. 16, no. 3, pp. 1834–1850, Mar. 2017.

[13] W. Xu, Y. Huang, W. Wang, F. Zhu, and X. Ji, "Toward ubiquitous and intelligent 6G networks: From architecture to technology," *Sci. China Inf. Sci.*, vol. 66, no. 3, pp. 5–6, Mar. 2023.

[14] M. Ke, Z. Gao, Y. Wu, X. Gao, and R. Schober, "Compressive sensing-based adaptive active user detection and channel estimation: Massive access meets massive MIMO," *IEEE Trans. Signal Process.*, vol. 68, pp. 764–779, 2020.

[15] Z. Gao et al., "Compressive-sensing-based grant-free massive access for 6G massive communication," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7411–7435, Mar. 2024.

[16] Y. Mei et al., "Compressive sensing-based joint activity and data detection for grant-free massive IoT access," *IEEE Trans. Wireless Commun.*, vol. 21, no. 3, pp. 1851–1869, Mar. 2022.

[17] Y. Li, M. Xia, and Y.-C. Wu, "Activity detection for massive connectivity under frequency offsets via first-order algorithms," *IEEE Trans. Wireless Commun.*, vol. 18, no. 3, pp. 1988–2002, Mar. 2019.

[18] X. Xu, X. Rao, and V. K. N. Lau, "Active user detection and channel estimation in uplink CRAN systems," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2015, pp. 2727–2732.

[19] Z. Chen, F. Sohrabi, and W. Yu, "Sparse activity detection in multi-cell massive MIMO exploiting channel large-scale fading," *IEEE Trans. Signal Process.*, vol. 69, pp. 3768–3781, 2021.

[20] D. Jiang and Y. Cui, "ML and MAP device activity detections for grant-free massive access in multi-cell networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 6, pp. 3893–3908, Jun. 2022.

[21] A. Fengler, S. Haghighatshoar, P. Jung, and G. Caire, "Non-Bayesian activity detection, large-scale fading coefficient estimation, and unsourced random access with a massive MIMO receiver," *IEEE Trans. Inf. Theory*, vol. 67, no. 5, pp. 2925–2951, May 2021.

[22] Z. Chen, F. Sohrabi, Y.-F. Liu, and W. Yu, "Phase transition analysis for covariance-based massive random access with massive MIMO," *IEEE Trans. Inf. Theory*, vol. 68, no. 3, pp. 1696–1715, Mar. 2022.

[23] D. Guo, L. Tang, X. Zhang, and Y. Liang, "Joint optimization of handover control and power allocation based on multi-agent deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13124–13138, Nov. 2020.

[24] Y. Qiang, X. Shao, and X. Chen, "A model-driven deep learning algorithm for joint activity detection and channel estimation," *IEEE Commun. Lett.*, vol. 24, no. 11, pp. 2508–2512, Nov. 2020.

[25] Y. Li, Z. Chen, Y. Wang, C. Yang, B. Ai, and Y.-C. Wu, "Heterogeneous transformer: A scale adaptable neural network architecture for device activity detection," *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 3432–3446, May 2023.

[26] A. Vaswani, "Attention is all you need," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, 2017, pp. 1–11.

[27] J. Dang, Z. Zhang, L. Wu, B. Zhu, and C. Li, "Joint channel estimation and active user detection for cell-free massive access system exploiting coarse user location information," *IEEE Internet Things J.*, early access, Dec. 12, 2024, doi: 10.1109/JIOT.2023.3345429.

[28] J. Li, H. Wang, P. Zhu, D. Wang, and X. You, "Covariance-based activity detection with orthogonal pilot sequences for cell-free distributed massive MIMO systems," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1307–1312, Jan. 2023.

[29] L. Diao, H. Wang, J. Li, P. Zhu, D. Wang, and X. You, "A scalable deep learning based active user detection approach for SEU-assisted cell-free massive MIMO systems," *IEEE Internet Things J.*, vol. 10, no. 22, pp. 19666–19680, Nov. 2023.

[30] M. Mossberg, E. K. Larsson, and E. Mossberg, "Estimation of large-scale fading channels from sample covariances," in *Proc. 45th IEEE Conf. Decis. Control*, Dec. 2006, pp. 2992–2996.

[31] J. Su, M. Zhu, A. Murtadha, S. Pan, B. Wen, and Y. Liu, "ZLPR: A novel loss for multi-label classification," 2022, *arXiv:2208.02955*.

[32] R. He, A. Ravula, B. Kanagal, and J. Ainslie, "RealFormer: Transformer likes residual attention," 2020, *arXiv:2012.11747*.

[33] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[34] M. Mohammadi, T. T. Vu, H. Q. Ngo, and M. Matthaiou, "Network-assisted full-duplex cell-free massive MIMO: Spectral and energy efficiencies," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 9, pp. 2833–2851, Sep. 2023.

[35] H. Q. Ngo, L.-N. Tran, T. Q. Duong, M. Matthaiou, and E. G. Larsson, "On the total energy efficiency of cell-free massive MIMO," *IEEE Trans. Green Commun. Netw.*, vol. 2, no. 1, pp. 25–39, Mar. 2018.

[36] H. Jiang, M. Cui, D. W. K. Ng, and L. Dai, "Accurate channel prediction based on transformer: Making mobility negligible," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 9, pp. 2717–2732, Sep. 2022.

[37] Y. Li, Q. Lin, Y.-F. Liu, B. Ai, and Y.-C. Wu, "Asynchronous activity detection for cell-free massive MIMO: From centralized to distributed algorithms," *IEEE Trans. Wireless Commun.*, vol. 22, no. 4, pp. 2477–2492, Apr. 2023.

[38] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[39] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," 2020, *arXiv:2006.04768*.

[40] D. Wang, M. Wang, P. Zhu, J. Li, J. Wang, and X. You, "Performance of network-assisted full-duplex for cell-free massive MIMO," *IEEE Trans. Commun.*, vol. 68, no. 3, pp. 1464–1478, Mar. 2020.

**Zheng Sheng** received the B.S. degree in communication engineering from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2020, and the M.Eng. degree in communication engineering from Southeast University, Nanjing, in 2022, where he is currently pursuing the Ph.D. degree in information and communication engineering. His research interests include full-duplex, massive MIMO, URLLC, and machine learning.

**Pengcheng Zhu** (Member, IEEE) received the Ph.D. degree in communication and information systems from Southeast University, Nanjing, China, in 2009. He is currently a Professor with the National Mobile Communications Research Laboratory, Southeast University. His research interests include wireless communications and mobile networks, including 5G/6G mobile communication systems, massive MIMO, ultra-reliable and low latency communications (URLLC), and mmWave communications.

**Xiaohu You** (Fellow, IEEE) received the master's and Ph.D. degrees in electrical engineering from Southeast University, Nanjing, China, in 1985 and 1988, respectively. Since 1990, he has been working with National Mobile Communications Research Laboratory, Southeast University, where he holds the rank of director and professor. His research interests include mobile communication systems, signal processing and its applications. He has contributed over 200 IEEE journal papers and two books in the areas of adaptive signal processing, neural networks and their applications to communication systems, and made important contributions to the development of China's 3G, 4G, and 5G mobile communications. From 1999 to 2002, he was the Principal Expert of the C3G Project, responsible for organizing China's 3G Mobile Communications Research and Development Activities. From 2001-2006, he was the Principal Expert of the China National 863 Beyond 3G FuTURE Project. Since 2013, he has been the Principal Investigator of China National 863 5G Project. Professor You served as the general chairs of IEEE WCNC 2013, IEEE VTC 2016 Spring and IEEE ICC 2019. Now he is Secretary General of the FuTURE Forum, vice Chair of China IMT-2020 Promotion Group, vice Chair of China National Mega Project on New Generation Mobile Network. Professor You was the recipient of the National 1st Class Invention Prize in 2011, and he was selected as IEEE Fellow in same year due to his contributions to development of mobile communications in China. He won the Chan Kah Kee Science Award in 2014 and IET Achievement Medal in 2021. He is a Member of Chinese Academy of Sciences.